Homomorphic Silent-setup Threshold Encryption from Lattices and Applications in Voting

Anonymous

Abstract. Hi

Keywords: Threshold Cryptosystem, Lattice-based Cryptography, Fully Homomorphic Encryption, Electronic Voting

1 Introduction

hello

1.1 Contributions

Here, I will bring the table that compares the classical ElectionGuard with distributed key generation vs. the Post-quantum Silent-setup (DKG-free) ElectionGuard based on my HSTE scheme.

Metric	ElectionGuard with DKG	DKG-free PQEG (Silent)	Improvement Factor
Setup Phase			
Time per guardian	$O(5{\cdot}3{\cdot}4096^2) \approx 10^{11}$ ops	$O(3{\cdot}2048^2{\cdot}60) \approx 10^9$ ops	$10^2\times$ **faster**
Space per guardian	$O(5{\cdot}3{\cdot}4096)$	$O(3{\cdot}2048{\cdot}60/8)$	**1.3**$\times$ **smaller**
Communication	$5^2{\cdot}3{\cdot}4096$	$5{\cdot}3{\cdot}2048{\cdot}60/8$	**1.3**$\times$ **less**
Interaction rounds	**2–3 rounds**	**0 rounds**	**Eliminates interaction**
Tally Phase			
Time per guardian	$O(5{\cdot}100{\cdot}4096^2) \approx 10^{12}$ ops	$O(100{\cdot}2048^2{\cdot}60) \approx 10^{10}$ ops	$10^2\times$ **faster**
Space per guardian	$O(5{\cdot}100{\cdot}4096)$	$O(100{\cdot}2048{\cdot}60/8)$	**1.3**$\times$ **smaller**
Communication per guardian	$5{\cdot}100{\cdot}4096$	$100{\cdot}2048{\cdot}60/8$	**1.4**$\times$ **less**
Total network communication	$5^2{\cdot}100{\cdot}4096$	$5{\cdot}100{\cdot}2048{\cdot}60/8$	**1.4**$\times$ **less**
Interaction rounds	**2 rounds**	**0 rounds**	**Eliminates interaction**

Table 1: Communication cost comparison between Classical ElectionGuard and PQEG

2 Related Works

2.1 Literature Review: Threshold Public Key Encryption

The Table 2 represents the summary of our survey, and Appendix **??** contains details of all of the techniques for designing threshold public key encryption schemes. Here,

we present our findings and observations, accompanied by a structural analysis of the techniques.

Chronological analysis reveals convergence toward specific design principles that show the temporal evolution of threshold public key encryption: Before 2010, schemes primarily utilized RSA-based assumptions (DCRA) with Paillier-type constructions. Between 2010 and 2020, the proliferation of pairing-based schemes with attribute-based extensions is tangible. After 2020, the dominance of lattice-based constructions with FHE capabilities suggests a shift toward post-quantum security.

Trade-off Patterns of Efficiency-Security A pattern emerges when examining the relationship between the complexity of decryption and the hardness assumptions. Schemes based on discrete logarithm problems (techniques [1], [2]) maintain $O(k^2)$ decryption complexity, while lattice-based constructions (techniques [3], [4], [5]) exhibit $O(n \cdot \log q)$ patterns with additional polynomial factors. The pairing-based schemes demonstrate a bifurcation: those employing bilinear maps directly (techniques [6], [7], [8]) achieve $O(t)$ to $O(t \cdot \log p)$ complexity, whereas schemes incorporating attribute-based structures (techniques [9], [10]) scale as $O(\ell^2)$ where ℓ represents the attribute universe size.

This complexity stratification correlates with the underlying algebraic structure rather than the security model. CCA-secure schemes do not uniformly exhibit higher complexity than their CPA counterparts - technique [11] achieves CCA security with $O(n+t^2)$ decryption while technique [12]'s CPA construction requires $O(nk|C|)$.

Dichotomy of Key Generation Further observations on Table 2 reveal two findings.

- Non-interactive schemes (techniques [13], [8]): These eliminate the distributed key generation phase entirely, achieving setup complexity independent of the number of parties. Technique [13]'s silent setup uses witness encryption to avoid interaction, while technique [8] employs a PKI-based approach with dynamic join capabilities.
- Distributed schemes (techniques [14], [15], [3], [4], [16], [5]): These require interactive protocols with communication complexity typically $O(n^2)$ for n parties. Within this category, a sub-pattern emerges where schemes using verifiable secret sharing (techniques [15], [16], [1]) incur additional zero-knowledge proof overhead compared to those using simple secret sharing.

The trusted dealer model (techniques [6], [12], [17], [18], [2], [19], [20], [21]) represents a middle ground, achieving a non-interactive setup at the cost of trust assumptions.

Ciphertext Expansion Patterns Size analysis of the ciphertext shows three distinct categories:

- Constant-size ciphertexts relative to threshold parameters: Techniques [8], [2], [22] achieve $O(|\mathbb{G}|)$ ciphertext size independent of t or n.

- Linear expansion: Techniques [9], [10] exhibit ciphertext growth with attribute set size or threshold value.
- Logarithmic expansion: FHE-based schemes (techniques [3], [4], [5]) demonstrate $O(\log q)$ growth where q relates to the noise management strategy.

Homomorphic Capability Stratification The threshold public key encryption schemes exhibit a clear stratification based on homomorphic properties.

Fully homomorphic (techniques [3], [4], [5], [23], [21], [24]): These universally employ noise management techniques - either modulus reduction (technique [3]) or noise flooding (techniques [5], [21]). The space complexity uniformly scales as $O(n \cdot L \cdot \text{ring size})$ where L represents the circuit depth or share dimension.

Linearly homomorphic (techniques [15], [16], [12], [25]): These maintain constant ciphertext size under addition but require key-switching for multiplication, suggesting an inherent limitation in achieving full homomorphism without ciphertext expansion.

Non-homomorphic (remaining techniques): These optimize for specific functionalities rather than computation on encrypted data.

Secret Sharing Methodology Classification The choice of a secret sharing scheme correlates strongly with the underlying algebraic structure, as Table 2 shows:

- Shamir-based sharing: Dominant in schemes over finite fields (18 out of 36 techniques)
- Additive sharing: Exclusive to lattice-based FHE schemes (techniques [3], [24])
- Linear Integer Secret Sharing (LISS): Appears only in schemes requiring adaptive security (techniques [14], [25])
- Polynomial-based sharing: Correlates with dynamic threshold capabilities (techniques [8], [17], [22])

Gap Analysis Our survey reveals redundancy in certain design spaces while exposing gaps in others. Pairing-based CPA-secure schemes with Shamir secret sharing represent 33% of the surveyed techniques, suggesting saturation in this design space, which indicates that these are over-explored. On the other hand, only technique [24] explores client-aided models, and only technique [13] achieves a truly silent setup, indicating unexplored potential in reducing setup assumptions, which these approaches are under-explored.

Classification by Structure A natural classification emerges based on the underlying algebraic structure:

- Group-based (techniques [13], [1], [2]): Rely on discrete logarithm in cyclic groups
- Ring-based (techniques [12], [17], [18], [19]): Utilize composite modulus arithmetic
- Lattice-based (techniques [14], [3], [4], [5], [23], [20], [21], [24]): Employ LWE/RLWE
- Pairing-based (techniques [6], [7], [8], [9], [10], [22], [26]): Require bilinear map
- Class group-based (techniques [15], [16], [25]): Operate in unknown order groups

This classification reveals that no single algebraic structure dominates across all performance metrics, suggesting fundamental trade-offs between different foundations.

Open Problems Based on this analysis, several non-trivial open problems emerge that can be considered as research directions and next forward steps.

- Optimal Complexity Lower Bounds: Determine whether $O(t)$ decryption complexity is achievable for a threshold t without using bilinear pairings. Current non-pairing schemes universally exhibit at least $O(t^2)$ complexity, but no formal barrier has been proven.
- Homomorphic Threshold Encryption without Noise Management: Is it possible to construct a fully homomorphic threshold public key scheme that eliminates the need for noise flooding or modulus reduction while maintaining security? All current FHE-based TPKE schemes require explicit noise management, suggesting a potential barrier.
- Dynamic Post-Quantum Threshold: Develop a TPKE scheme supporting dynamic threshold adjustment (varying t post-setup) while maintaining post-quantum security. Current dynamic schemes (techniques [8], [22]) rely exclusively on discrete logarithm or pairing assumptions.

2.2 Silent-setup Threshold Public Key Schemes

3 Preliminaries

3.1 Computational Assumptions

Module Short Integer Solution (MSIS) [111] Let $n,k,m,q,\beta \in \mathbb{N}$ where n is a power of 2, and let $R=\mathbb{Z}[x]/(x^n+1)$ be the n-th cyclotomic ring with $R_q=R/qR$. The Module Short Integer Solution problem $\mathsf{MSIS}_{n,k,m,q,\beta}$ is defined as follows: given a uniformly random matrix $\mathbf{A} \leftarrow\$ R_q^{k\times m}$, find a non-zero vector $\mathbf{z} \in R^m$ such that $\mathbf{A}\cdot\mathbf{z}=\mathbf{0} \in R_q^k$ and $\|\mathbf{z}\|_2 \leq \beta$, where $\|\mathbf{z}\|_2 = \sqrt{\sum_{i=1}^{m}\|\mathbf{z}_i\|_2^2}$ is the Euclidean norm of the coefficient vector concatenation. An algorithm $\mathcal{A}$ solves $\mathsf{MSIS}_{n,k,m,q,\beta}$ with advantage ϵ if $\Pr\left[\mathbf{A}\leftarrow\$ R_q^{k\times m}; \mathbf{z}\leftarrow\mathcal{A}(\mathbf{A}) : \mathbf{A}\cdot\mathbf{z}=\mathbf{0}\wedge\mathbf{z}\neq\mathbf{0}\wedge\|\mathbf{z}\|_2\leq\beta\right]\geq\epsilon$.

Module Learning with Errors (MLWE) [112] Let $n,k,m,q \in \mathbb{N}$ where n is a power of 2, and let $R=\mathbb{Z}[x]/(x^n+1)$ be the n-th cyclotomic ring with $R_q=R/qR$. Let χ be a distribution over R_q. Consider $D_0:=(\mathbf{A},\mathbf{A}^T\mathbf{s}+\mathbf{e})$ and $D_1:=(\mathbf{A},\mathbf{u})$, where $\mathbf{A}\leftarrow\$ R_q^{k\times m}, \mathbf{s}\leftarrow\$ R_q^k, \mathbf{e}\leftarrow\$ \chi^m, \mathbf{u}\leftarrow\$ R_q^m$. The $\mathsf{MLWE}(n,k,m,q,\chi)$ problem is to decide between the distributions D_0 and D_1. The module-LWE problem generalizes both LWE (when $n=1$) and RLWE (when $k=1$), and its hardness is based on worst-case lattice problems over module lattices.

Ring Learning with Errors (RLWE) [113] Let $n,m,q \in \mathbb{N}$ where n is a power of 2, and let $R=\mathbb{Z}[x]/(x^n+1)$ be the n-th cyclotomic ring with $R_q=R/qR$. Let χ be a distribution over R_q. Consider $D_0 := (\mathbf{a}, \mathbf{a}\cdot s+e)$ and $D_1 := (\mathbf{a}, u)$, where $\mathbf{a}\leftarrow\$ R_q^m, s\leftarrow\$ R_q, e\leftarrow\$ \chi^m, u\leftarrow\$ R_q^m$. The $\mathsf{RLWE}(n,m,q,\chi)$ problem is to decide between the distributions D_0 and D_1. Furthermore, if the error distribution χ is bounded by a parameter B_{rlwe}, then solving $\mathsf{RLWE}(n,m,q,\chi)$ is at least as hard as approximating worst-case problems over ideal lattices in the ring R within the factor of $\tilde{O}\left(n\cdot q/B_{\mathsf{rlwe}}\right)$.

Technique \ Property	Construction Method	Time Complexity	Space Cost	Secret Key Size	Public Key Size	Ciphertext Size	Hardness Assumption	Security Property	Key Generation	Sharing Type	Implementation	Core Feature
2. Crypto 24 [13]	TS [27] + Com [28] + IP [29] + WE [13]	Enc $O(1)$ Dec $O(T^3)$	$(M+2)\lvert\mathbb{G}_1\rvert+(M+1)\lvert\mathbb{G}_2\rvert+$ $(M+3)\lvert U\rvert\lvert\mathbb{G}_1\rvert+2\lvert\mathbb{G}_1\rvert+$ $8\lvert\mathbb{G}_2\rvert+\lvert\mathbb{G}_T\rvert+\lvert\mathbb{Z}_p^*\rvert$	$\lvert\mathbb{Z}_p\rvert$	$\lvert\mathbb{G}_1\rvert$	$2\lvert\mathbb{G}_1\rvert+2\lvert\mathbb{G}_2\rvert+$ $\lvert\mathbb{G}_T\rvert$	q-polynomial	IND-CPA secure in the Generic Group Model	Non-interactive	No sharing (silent)	✓	DMG-free; removes costly interactive setup
3. PKC 21 [14] Scheme 1	HPS [16] + NIZK [11] + LSS [12] + PKE [10]	Enc $O(m \cdot \mathsf{nbcrp} + L(m+n)\log q)$ Dec $O(d \cdot \mathsf{nbcrp} + L \cdot \log q)$	$2\lvert\mathbb{Z}^*_{N^{s+1}}\rvert+2\lvert\mathbb{Z}_q\rvert+d\cdot\lvert\mathbb{G}+$ $\lvert\mathsf{CRS}_{\mathsf{nizk}}\rvert$, s.t. $d=O(t^{1+\frac{1}{s}})$	$d_i \cdot \lvert\mathbb{G}\rvert$	$\lvert\mathbb{Z}_{N^{s+1}}\rvert+\lvert\mathbb{G}_{\mathsf{pke}}\rvert$ $+\lvert\mathsf{CRS}_{\mathsf{pke}}\rvert$	$2\lvert\mathbb{Z}^*_{N^{s+1}}\rvert+\lvert\mathbb{G}_{\mathsf{pke}}\rvert$	DCR and LWE	IND-CCA2 under adaptive corruptions in the standard model	Distributed	LSS	✗	public verifiability and voting-friendly downgrade
3. PKC 21 [14] Scheme 2	PKE [10] + LSS [12] + NIZK [11]	Enc $O(m \cdot \mathsf{nbcrp} + L(m+n)\log q)$ Dec $O(d \cdot \mathsf{nbcrp} + L \cdot \log q)$	$2\lvert\mathbb{Z}^{s+1}_N\rvert+2\lvert\mathbb{Z}^{s+1}_N\rvert+d\cdot\lvert\mathbb{Z}^{s+1}_N\rvert$ $+\lvert\mathsf{CRS}_{\mathsf{nizk}}\rvert$, s.t. $d=O(t^{1+\frac{1}{s}})$	$d_i \cdot \lvert\mathbb{Z}^{s+1}_N\rvert$	$2\lvert\mathbb{Z}^{s+1}_N\rvert+2\lvert\mathbb{Z}^{s+1}_N\rvert$ $+\lvert\mathsf{CRS}_{\mathsf{nizk}}\rvert$	$2\lvert\mathbb{Z}^{s+1}_N\rvert+\lvert\mathbb{G}_{\mathsf{pke}}\rvert$	LWE	IND-CCA2 under adaptive corruptions in the standard model	Distributed	LSS	✗	decryption with reduced interaction
4. SCN 24 [15]	HE [15] + FE [36] + VSS$_1$ [37] + ZKP [38] + VSS$_2$ [39]	Enc $O(1)$ Dec $O(N \cdot \log^3(s) + t \cdot \log^2(s))$	$O(N \cdot \log(s) \cdot \lvert\mathbb{F}_p\rvert + t \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{F}\rvert)$	$O(\log(s))$ bits per party	$O(\lvert\mathbb{G}\rvert)$	$O(\lvert\mathbb{G}\rvert+\lvert\mathbb{F}\rvert)$	HSM, ORD, ROC	IND-CPA UC-secure (static and adaptive SIP)	Distributed	Shamir Secret Sharing	✓	Efficiency via PoK removal; transparent setup
5. Asiacrypt 24 [3]	FHE [3] + SS [40] + GR [41] + NF [41]	Enc $O(m \cdot n \cdot \log Q)$ Dec $O(n \cdot \log_{\mathsf{msg}} + N \cdot n \cdot \log_{\mathsf{msg}})$	$(m \cdot n + m) \cdot \lvert\mathbb{Z}_q\rvert + (m+1) \cdot \lvert\mathbb{Z}_{\mathsf{gen}}\rvert + N \cdot n \cdot \lvert\mathbb{Z}_{\mathsf{gen}}\rvert$	$n \cdot \lvert\mathbb{Z}_{\mathsf{gen}}\rvert$	$(m \cdot n + m) \cdot \lvert\mathbb{Z}_q\rvert$	$(n+1) \cdot \lvert\mathbb{Z}_q\rvert$	piLWE	IND-CPA	Distributed (N-out-of-N)	Additive Secret Sharing	✗	Low communication via modulus reduction
6. ISC 24 [4]	FHE [42] + SS [43] + LR [44]	Enc $O(d \cdot t \cdot \log q)$ Dec $O(d \cdot \log q)$	$O(n \cdot t \cdot \lvert R_q\rvert \text{ s.t. } \lvert R_q\rvert \sim d \cdot \log q$	$\lvert R_q\rvert$	$2 \cdot t \cdot \lvert R_q\rvert$	$2\lvert R_q\rvert$	RLWE	IND-CPA, UC-secure	Distributed	Shamir Secret Sharing	✗	Robustness in refinestration
7. Crypto 23 [16]	VSS [45] + ZKP [16, 46] + PKE [47, 48]	Enc $O(\log q)$ Dec $O(\cdot N \log N * \log q)$	$O(N \cdot \lvert\mathbb{G}_q\rvert + \lvert\mathbb{G}\rvert + \lvert\mathbb{F}\rvert)$	$O(\lvert\mathbb{G}_q\rvert + \lvert\mathbb{G}_p\rvert)$	$O(\lvert\mathbb{G}_q\rvert)$	$O(\lvert\mathbb{G}_q\rvert)$	HSM and ORD	IND-CPA UC-secure (static and adaptive SIP)	Distributed	Shamir Secret Sharing	✓	Efficiency via PoK removal; transparent setup
8. Asiacrypt 23 [5]	LSS [49] + FHE [50] + NF [51]	Enc $O(\cdot d \cdot \log q)$ Dec $O(\alpha \cdot L \cdot t \cdot d \cdot \log q + \alpha_{\mathsf{ms}} \cdot d \cdot \log q)$	$O(\alpha \cdot L \cdot \lvert\mathbb{G}_1^{\mathsf{ms}}\rvert + \lvert\mathbb{G}_1^{\mathsf{ms}}\rvert)$	$O(L \cdot \lvert\mathbb{G}_1^{\mathsf{ms}}\rvert)$	$O(\beta^{s}+\epsilon) \cdot \lvert\mathbb{G}_q\rvert K\rvert/(K^s+1))$	$O((\beta^{s}+\epsilon) \cdot \lvert\mathbb{G}_q\rvert K\rvert/(K^s+1))$	MLWE	Selective IND-CPA (from OW-CPA)	Distributed (t-out-of-n)	Replicated Shamir Secret Sharing	✗	Noise-analyzed threshold decryption
10. PKC 04 [6]	BM [52] + SS [53] + PKE [54] + ZKP [55]	Enc $O(1)$ Dec $O(t)$	$(n+4)\lvert\mathbb{G}\rvert+n\lvert\mathbb{Z}\rvert+t$	$\lvert\mathbb{G}\rvert$ per server	$\lvert\mathbb{G}\rvert+n\lvert\mathbb{Z}\rvert$	$2\lvert\mathbb{G}\rvert+t$	ROM	IND-CCA in ROM	Distributed (PKG + DID)	Shamir Secret Sharing	✗	Supports mediated encryption
12. USENIX 24 [7]	Com [56] + NIZK [57] + SS [58] + BM [59]	Enc $O(1)$ Dec $O(Bn_B\beta)$	$O(B \cdot \lvert\mathbb{G}_1\rvert + B \cdot \lvert\mathbb{G}_2\rvert + n \cdot B \cdot \lvert\mathbb{F}\rvert)$	$O(B \cdot \lvert\mathbb{F}\rvert)$	$O(B \cdot (\lvert\mathbb{G}_1\rvert+\lvert\mathbb{G}_2\rvert))$	$2\lvert\mathbb{G}_2\rvert+\lvert\mathbb{G}_1\rvert+3\lvert\mathbb{F}\rvert+\lvert M\rvert+\lvert\mathsf{NIZK}\rvert$	q-SHDHT (pairing-friendly groups)	IND-CCA2 in ROM	Distributed (trusted dealer)	Shamir Secret Sharing	✓	Independence of batch size
16. Eurocrypt 04 [12]	HE [60] + ZKP [61] + SS [62]	Enc $O(k^3)$ Dec $O(ckk'l')$	$O(n \cdot \lvert\mathbb{Z}_{n^2}\rvert + n \cdot \lvert\mathbb{Z}_{n'}\rvert)$	$\lvert\mathbb{Z}_{n'}\rvert$	$\lvert\mathbb{Z}_{n^2}\rvert$	DCR	UC-secure against active (static) adversaries	Distributed (trusted dealer)	Integer Shamir Secret Sharing	✓	VSS fully overhead-free MPC
17. Crypto 08 [8]	BM [63] + PI [64] + KEM [65]	Enc $O(m+t+s)$ Dec $O(\cdot \log r \cdot m)$	$O(m \cdot \lvert\mathbb{G}_n\rvert + t \cdot \lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_2\rvert + m \cdot \lvert\mathbb{Z}_p\rvert)$	$\lvert\mathbb{G}_1\rvert$	$\lvert\mathbb{Z}_p\rvert$	$\lvert\mathbb{G}_1\rvert+\lvert\mathbb{G}_2\rvert+s \cdot \lvert\mathbb{Z}_p\rvert$	MSE−DDH (bilinear groups)	IND-CPA in the standard model	PKD-based with dynamic join	Polynomial-based	✗	Dynamic authorized set
18. Eurocrypt '91 [1]	VSS [66] + PKE [67] + Com [68]	Enc $O(1)$ Dec $O(k^3)$	$O(nk)\lvert\mathbb{G}_n\rvert+O(nk)\lvert\mathbb{Z}_{\mathsf{ord}}\rvert$	$\lvert\mathbb{Z}_{\mathsf{ord}}\rvert$	$\lvert\mathbb{G}_n\rvert$	$2\lvert\mathbb{G}_n\rvert$	DLP	IND-CPA in the standard model	Distributed (no trusted party)	Shamir Secret Sharing	✗	Elimination of a trusted key-distribution
19. ACISP 03 [17]	SS [69] + PI [70] + PKE [71] + ZKP [72]	Enc $O(\sqrt{N^3})$ Dec $O(\sqrt{N^3}+\sqrt{N^3})$	$O(n\lvert\mathbb{Z}_n\rvert+O(1)\lvert\mathbb{Z}_{n+1}\rvert$	$\lvert\mathbb{Z}_{\mathsf{ord}}\rvert$	$O(1)\lvert\mathbb{Z}_n\rvert$	$\lvert\mathbb{Z}_n\rvert+\lvert\mathbb{Z}_{n+1}\rvert$	DCRA, and Composite DDH	IND-CPA secure in ROM	Distributed	Polynomial-based	✗	Length-flexible, fixed-size threshold
20. Crypto 03 [18]	TS [73] + SS [74] + ZKP [73]	Enc $O(\log^2 N)$ Dec $O(t \cdot \log^2 N + t^2 \cdot \log N)$	$O(t \cdot \lvert\mathbb{Z}_n\rvert + \lvert\mathbb{Z}_{n^2}\rvert)$	$\lvert\mathbb{Z}_{n^2}\rvert + \lvert\mathbb{Z}_{\mathsf{ord}}\rvert$	$+\lvert\mathbb{Z}_{\mathsf{ord}}\rvert$	$\lvert\mathbb{Z}_{n^2}\rvert$	DCRA	UC-secure against active (adaptive) adversaries	Distributed	Polynomial-based	✗	Avoids non-committing encryption
21. Asiacrypt 22 [25]	CC [76] + LSS [77, 78] + PKE [79]	Enc $O(k+\log l)$ Dec $O(k^3 \cdot t \log l)$	$2\lvert\mathbb{CL}_0\rvert+n\cdot\lvert\mathbb{Z}_0^{\mathsf{Secu}}$	$\lvert\mathbb{Z}_0^{\mathsf{Sk}}$	$\lvert\mathbb{CL}_0\rvert$	$2\cdot\lvert\mathbb{CL}_0\rvert$	HSM$_{\mathsf{gr}}$	IND-CPA in the standard model	Distributed,/ Non-threshold procedure	Linear Integer Secret Sharing	✓	Efficient homomorphic evaluation
23. Eurocrypt '99 [11]	SS [80] + PKE [81] + H [82]	Enc $O(1)$ Dec $O(n+t^2)$	$O(L \cdot \lvert\mathbb{Z}_n\rvert + \lvert\mathbb{G}\rvert)$	$(5+2L)\cdot\lvert\mathbb{Z}_n\rvert$	$O(\lvert\mathbb{G}\rvert)$	$4\cdot\lvert\mathbb{G}$	DDH	Adaptive CCA-secure in the standard model	Distributed	Shamir Secret Sharing	✗	eliminating the need for interactive validity checks
25. ACISP '12 [9] Scheme 1	BM [63] + SS [84] + ABE [85]	Enc $O(1)$ Dec $O(t^3+1)$	$O(\cdot\lvert\mathbb{G}\rvert+\ell+\lvert\mathbb{G}\rvert+\lvert\mathbb{G}\rvert+\lvert\mathbb{G}\rvert+\lvert\mathbb{Z}_0\rvert)$	$2(\ell+s)\cdot\lvert\mathbb{G}\rvert$	$(2\ell+2)\cdot\lvert\mathbb{G}\rvert+\lvert\mathbb{G}\rvert$	$\lvert\mathbb{G}\rvert+2\lvert\mathbb{G}\rvert$	q − BDHE	CPA selectively secure	Central	Polynomial-based	✗	flexible threshold access structures
25. ACISP '12 [9] Scheme 2	BM [63] + SS [84] + ABE [85] + H [86]	Enc $O(1)$ Dec $O(t^3+1)$	$O(\cdot\lvert\mathbb{G}\rvert+\ell+\lvert\mathbb{G}\rvert+\lvert\mathbb{G}\rvert+\lvert\mathbb{G}\rvert+\lvert\mathbb{Z}_0\rvert)$	$2(\ell+s)\cdot\lvert\mathbb{G}\rvert$	$(2\ell+2)\cdot\lvert\mathbb{G}\rvert+\lvert\mathbb{G}_T\rvert$	$\lvert\mathbb{G}_T\rvert+3\lvert\mathbb{G}\rvert+\lvert\mathbb{Z}_0\rvert$	q − BDHE	CCA in standard model	Central	Polynomial-based	✗	flexible threshold access structures
26. Inf. Sci. 10 [10]	DMG [87] + SS [88] + BM [89]	Enc $O(A_0 \cdot (\log r + n)$ Dec $O((t+1) \cdot d_0 \cdot \log q + (t+1) \cdot 3)$	$O(n \cdot m \cdot \lvert\mathbb{Z}_q\rvert + n \cdot N \cdot \lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_2\rvert)$	$O(A_0 \cdot \lvert\mathbb{G}_2\rvert + \log q)$	$O(N \cdot \lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_2\rvert)$	$O(\lvert A_0\rvert \cdot \lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_1\rvert + \lvert M\rvert)$	DBDH	Selectively secure CPA in the static adversary model	Distributed	Joint Zero-Secret Sharing	✗	N/A
27. ICICS 10 [2]	BM [90] + SS [91]	Enc $O(\lvert R\rvert+1)$ Dec $O(t)$	$O(n \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{G}_2\rvert)$	$O(n \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{Z}_p\rvert)$	$O(n \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{G}_T\rvert)$	$O(\lvert\mathbb{G}\rvert)$	DBDHE	IND-CPA secure under adaptive corruption	Centralized (Trusted dealer)	Shamir Secret Sharing	✗	constant-size ciphertexts with adaptive security without random oracles
28. ACISP 16 [19]	SS [92] + PKE [93] + ZKP [94]	Enc $O(t^2)$ Dec $O(t^2)$	$O(n \cdot \lvert\mathbb{Z}_{n^2}\rvert + n \cdot t \cdot \log N + \lvert\mathbb{Z}_{N^2}\rvert)$	$O(\lvert\mathbb{Z}_{n\phi}\rvert)$	$O(\lvert\mathbb{Z}_{n\phi}\rvert + n \cdot \lvert\mathbb{Z}_{n^2}\rvert + n \cdot t \cdot \log N)$	$O(\lvert\mathbb{Z}_{n^2}\rvert)$	DCRA	Threshold semantic security against static adversaries in ROM	Centralized (Trusted dealer)	Blakely's Secret Sharing	✗	Smaller exponents; avoids inverses
29. PQCrypto 11 [20]	TBE [95] + SS [96] + OTS [97]	Enc $O(n \cdot \gamma^2 \log^6 n + n \cdot \gamma^2 \log n)$ Dec $O(\gamma^2 \log n + n \cdot \gamma^2 \log n)$	$O(n \cdot \gamma^2 \log n \lvert\mathbb{Z}_n^*\rvert$	$\gamma^2 \log n \lvert\mathbb{Z}_n^*\rvert$	$2n\gamma^2 \log n \lvert\mathbb{Z}_n^*\rvert$	$2n\gamma^2 \log n \lvert\mathbb{Z}_n^*\rvert$	SIVP	Selective-tag CCA security in the standard model	Centralized (Trusted dealer)	Generic Secret Sharing	✗	Selective-tag security; lossy trapdoors
30. COLING. 22 [21]	HE [98, 99] + LSS [100] + NF [101]	Enc $O(k\log q)$ Dec $O(t^2 n \log q + tn)$	$O(t\lvert\mathbb{G}\rvert + t\lvert\mathbb{F}_q\rvert)$	$O(\lvert\mathbb{F}_q\rvert)$	$O(\lvert\mathbb{F}_q\rvert)$	$O(\lvert\mathbb{G}_1\rvert)$	RLWE	IND- CPA	Centralized (Trusted dealer)	$\{0,1\}$-LSS	✗	collusion-attacks-free noise flooding
34. ESORICS 23 [24]	CAM [102] + SS [34, 103] + FHE [104] + MUX [24]	Enc $O(\kappa N^2\binom{N}{t}+N\lvert T\rvert)$ Dec $O(\kappa N^2\binom{N}{t-1}+N\lvert T\rvert)$	$O(n \cdot \lvert T\rvert \cdot N + kN \cdot (T_{\mathsf{ct}}\lvert X\rvert \cdot N + (k+1)\lvert \cdot (T_{\mathsf{ct}}\lvert X\rvert) \cdot N)$	$O(n \cdot \lvert\mathbb{R}\rvert$	$O(kN\lvert\mathsf{ct}\rvert+1) \cdot \lvert T\rvert + \kappa(k+1)lN \cdot (T_{\mathsf{ct}}\lvert X\rvert))$	$O((k+1) \cdot (T_{\mathsf{ct}}\lvert X\rvert))$	TRSWE	IND-CPA under semi-honest adversaries	Distributed	Additive Secret Sharing	✓	Client-aided noise; efficient bootstrapping
35. Inf. Sci. 18 [23]	BM [105] + PC [22, 106] + PI [107]	Enc $O(n+3)$ Dec $O(t^3+s)$	$(2m+2+\lvert A\rvert)\lvert\mathbb{G}\rvert+\lvert\mathbb{Z}_p\rvert$	$((A+m)\lvert\mathbb{G}\rvert$	$(2m+2)\lvert\mathbb{G}\rvert$	$2\lvert\mathbb{G}\rvert+\lvert\mathbb{G}_T\rvert$	aMSE−DDH	IND-mCPA in the standard model	Centralized (Trusted dealer)	Polynomial-based	✗	Dynamic threshold
36. CT-RSA 06 [26]	IBE [26, 108] + CHK [109] + DS [110]	Enc $O(1)$ Dec $O(k^3)$	$O(n \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{G}_1\rvert + \lvert\mathbb{Z}_p\rvert)$	$\lvert\mathbb{G}\rvert$	$(n+4) \cdot \lvert\mathbb{G}\rvert + \mathsf{max}$	$2 \cdot \lvert\mathbb{G}\rvert + \lvert\mathbb{G}_1\rvert + \mathsf{NmK}\rvert + \lvert\sigma\rvert$	DBDH	CCA2 secure in the standard model	Centralized (Trusted dealer)	Polynomial-based	✗	Public verification; oracle-free threshold decryption

Table 2: Survey Summary of Threshold Public Key Schemes. Complexity measures employ standard asymptotic notation where $O(\cdot)$ denotes worst-case upper bounds expressed in terms of elementary operations. Time complexities distinguish between the encryption (Enc) and decryption (Dec) phases, with costs measured in terms of group operations, field arithmetic, or pairing computations, depending on the underlying primitive. Space complexities and key/ciphertext sizes are expressed using bit-length notations: $|\mathbb{G}_1|$, $|\mathbb{G}_2|$, $|\mathbb{G}_T|$ represent the size of elements in bilinear pairing groups (source groups and target group respectively); $|\mathbb{Z}_p|$, $|\mathbb{Z}_q|$, $|\mathbb{Z}_N|$ denote sizes of elements in finite fields or integer rings modulo primes p, q, or composite N; and $|\mathbb{F}|$ indicates generic field element sizes. Common parameters include: n (number of parties or lattice dimension), t (threshold parameter—minimum parties needed for decryption), k (security parameter or polynomial degree), d (dimension or number of shares), m (message space dimension or maximum attribute set size), q (modulus or prime order), λ (computational security parameter), ℓ (attribute universe size or gadget decomposition parameter), N (RSA modulus or total parties), s (flexibility parameter or attribute set size), B (batch size or decomposition base), r (RLWE rank or randomness dimension), L (circuit depth or share vector length), and $|U|$ (universe/committee size). Polynomial factors like $\log q$, $\log N$ account for bit-complexity of arithmetic operations, while terms such as $\mathsf{poly}(\lambda)$ indicate polynomial dependence on the security parameter. "✓" indicated that there exists an implementation of the threshold design technique, and "✗" specifies there is no source code or an accessible implementation.

Gap-Strong Computational Diffie-Hellman (Gap-StCDH) [114] Let $\mathbb{G}$ be a cyclic group of prime order p with generator g. The Strong Computational Diffie-Hellman (StCDH) problem is: given $(g,g^a,g^b,g^c) \in \mathbb{G}^4$ where $a,b \leftarrow\$ \mathbb{Z}_p$ and $c \leftarrow\$ \mathbb{Z}_p$, determine whether $c = ab \pmod p$. The Gap-StCDH problem allows the adversary access to a Decisional Diffie-Hellman (DDH) oracle that, on input (g^x,g^y,g^z), returns 1 if $z = xy \pmod p$ and 0 otherwise. An adversary has advantage ϵ in solving Gap-StCDH if it correctly decides the StCDH instance with probability at least $1/2+\epsilon$.

3.2 Building Blocks

Adapted from Boneh et al. [115], the following is the definition of the threshold public key encryption scheme, along with its properties.

Definition 1 (Random Oracle [116]). *A hash function $H : \{0,1\}^* \to \{0,1\}^\ell$ is modeled as a random oracle if:*

- *On each new query $x \in \{0,1\}^*$, $H(x)$ is sampled uniformly from $\{0,1\}^\ell$*
- *On repeated queries to the same x, $H(x)$ returns the previously sampled value*
- *All parties (including adversaries) have oracle access to $H(\cdot)$*

We heuristically instantiate H using SHA-256 *with output length $\ell = 256$ bits, or* SHA3-256 *for domain separation when multiple independent hash functions required.*

Definition 2 (Threshold PKE). *Let $\mathcal{P} = \{P_1,...,P_N\}$ denote a set of parties and $\mathbb{S}$ denote a class of efficient access structures over $\mathcal{P}$. A* threshold public-key encryption scheme *for message space $\mathcal{M}$ and access structure class $\mathbb{S}$ consists of the following polynomial-time algorithms:*

- $\mathsf{Setup}(1^\lambda,\mathbb{A}) \to (pp,ek,sk_1,...,sk_N)$*: Takes as input the security parameter λ and an access structure $\mathbb{A} \in \mathbb{S}$, and outputs public parameters pp, an encryption key ek, and key shares $sk_1,...,sk_N$.*
- $\mathsf{Enc}(ek,m) \to ct$*: Takes as input the encryption key ek and a message $m \in \mathcal{M}$, and outputs a ciphertext ct.*
- $\mathsf{PartDec}(pp,sk_i,ct) \to m_i$*: Takes as input the public parameters pp, a key share sk_i, and a ciphertext ct, and outputs a partial decryption share m_i.*
- $\mathsf{PartVerify}(pp,ct,m_i) \to \{0,1\}$*: Takes as input the public parameters pp, a ciphertext ct, and a partial decryption share m_i, and outputs accept (1) or reject (0).*
- $\mathsf{Combine}(pp,\mathcal{B}) \to m'$*: Takes as input the public parameters pp and a set of partial decryption shares $\mathcal{B} = \{m_i\}_{i \in S}$, and outputs a message m'.*

Definition 3 (Compactness). *A threshold* PKE *scheme for access structure class $\mathbb{S}$ satisfies* compactness *if there exist polynomials $\mathsf{poly}_1(\cdot)$ and $\mathsf{poly}_2(\cdot)$ such that for all security parameters λ and access structures $\mathbb{A} \in \mathbb{S}$, whenever $(pp,ek,sk_1,...,sk_N) \leftarrow \mathsf{Setup}(1^\lambda,\mathbb{A})$ and $ct \leftarrow \mathsf{Enc}(ek,m)$ for any message m, it holds that:*

$$|ct| \le \mathsf{poly}_1(\lambda) \quad \textit{and} \quad |ek| \le \mathsf{poly}_2(\lambda).$$

Definition 4 (Decryption Correctness). *A threshold* PKE *scheme for access structure class* $\mathbb{S}$ *satisfies* decryption correctness *if for all security parameters* λ*, access structures* $\mathbb{A}\in\mathbb{S}$*, authorized sets* $S\in\mathbb{A}$*, and messages* $m\in\mathcal{M}$*, the following probability is overwhelming:*

$$\Pr\left[\begin{array}{l}(pp,ek,sk_1,...,sk_N)\leftarrow\mathsf{Setup}(1^\lambda,\mathbb{A});\quad ct\leftarrow\mathsf{Enc}(ek,m);\\ \{m_i\leftarrow\mathsf{PartDec}(pp,sk_i,ct)\}_{i\in S};\quad m'\leftarrow\mathsf{Combine}(pp,\{m_i\}_{i\in S});\\ \textbf{return }(m'=m)\end{array}\right]\geq 1-\mathsf{negl}(\lambda).$$

Definition 5 (Partial Verification Correctness). *A threshold* PKE *scheme for access structure class* $\mathbb{S}$ *satisfies* partial verification correctness *if for all security parameters* λ*, access structures* $\mathbb{A}\in\mathbb{S}$*, authorized sets* $S\in\mathbb{A}$*, parties* $i\in S$*, and messages* $m\in\mathcal{M}$*, the following probability is overwhelming:*

$$\Pr\left[\begin{array}{l}(pp,ek,sk_1,...,sk_N)\leftarrow\mathsf{Setup}(1^\lambda,\mathbb{A});\quad ct\leftarrow\mathsf{Enc}(ek,m);\\ m_i\leftarrow\mathsf{PartDec}(pp,sk_i,ct);\quad b\leftarrow\mathsf{PartVerify}(pp,ct,m_i);\\ \textbf{return }(b=1)\end{array}\right]\geq 1-\mathsf{negl}(\lambda).$$

Definition 6 (Security). *A threshold* PKE *scheme for access structure class* $\mathbb{S}$ *satisfies* security *(IND-CCA) if for any efficient adversary* $\mathcal{A}$*, the advantage*

$$\mathsf{Adv}^{\mathsf{TPKE}}_{\mathcal{A}}(\lambda)=\left|\Pr[\mathsf{Expt}_{\mathcal{A},0}(1^\lambda)=1]-\Pr[\mathsf{Expt}_{\mathcal{A},1}(1^\lambda)=1]\right|$$

is negligible in λ*, where experiment* $\mathsf{Expt}_{\mathcal{A},b}(1^\lambda)$ *is defined as Figure 1.*

Definition 7 (Robustness). *A threshold* PKE *scheme for access structure class* $\mathbb{S}$ *satisfies* robustness *if for any efficient adversary* $\mathcal{A}$ *and any security parameter* λ*, the following probability is negligible:*

$$\Pr\left[\begin{array}{l}\mathbb{A}\leftarrow\mathcal{A}(1^\lambda);\quad (pp,ek,sk_1,...,sk_N)\leftarrow\mathsf{Setup}(1^\lambda,\mathbb{A});\\ (ct^*,m_i^*,i^*)\leftarrow\mathcal{A}(pp,ek,sk_1,...,sk_N);\quad m'_{i^*}\leftarrow\mathsf{PartDec}(pp,sk_{i^*},ct^*);\\ b_1\leftarrow\mathsf{PartVerify}(pp,ct^*,m_i^*);\quad b_2\leftarrow(m_i^*\neq m'_{i^*});\quad \textbf{return }(b_1\wedge b_2)\end{array}\right]\leq\mathsf{negl}(\lambda).$$

Definition 8 ([117]). *The* BFV *is a fully homomorphic encryption scheme, based on* RLWE *with parameters* $(n,q,p,\chi_{key},\chi_{err})$ *where* $R=\mathbb{Z}[X]/(X^n+1)$*,* $R_q=R/qR$*,* $R_p=R/pR$*, and* $\Delta=\lfloor q/p\rfloor$*, consists of* $\mathsf{BFV}=(\mathsf{KeyGen},\mathsf{Enc},\mathsf{Dec},\mathsf{Add},\mathsf{Mult},\mathsf{RelinKeyGen})$.

- $(pk,sk,\mathsf{rlk})\leftarrow\mathsf{KeyGen}(1^\lambda)$*: On input security parameter* 1^λ*, samples secret key* $s\leftarrow\chi_{key}$ *over* R*, samples* $\mathbf{a}\leftarrow R_q^k$ *uniformly and error* $\mathbf{e}\leftarrow\chi_{err}^k$*, computes public key* $\mathbf{b}=-(\mathbf{a}\cdot s+\mathbf{e})\in R_q^k$*, generates relinearization key* rlk *for* s^2*, and outputs* $pk=(\mathbf{a},\mathbf{b})$*,* $sk=s$*, and* rlk*.*
- $ct\leftarrow\mathsf{Enc}(pk,\mu)$*: Takes input public key* $pk=(\mathbf{a},\mathbf{b})$ *and plaintext* $\mu\in R_p$*, samples* $\mathbf{u}\leftarrow\chi_{key}$*,* $e_1,e_2\leftarrow\chi_{err}$*, computes* $ct_0=\mathbf{b}\cdot\mathbf{u}+e_1+\Delta\cdot\mu\in R_q$ *and* $ct_1=\mathbf{a}\cdot\mathbf{u}+e_2\in R_q$*, and outputs* $ct=(ct_0,ct_1)$*.*

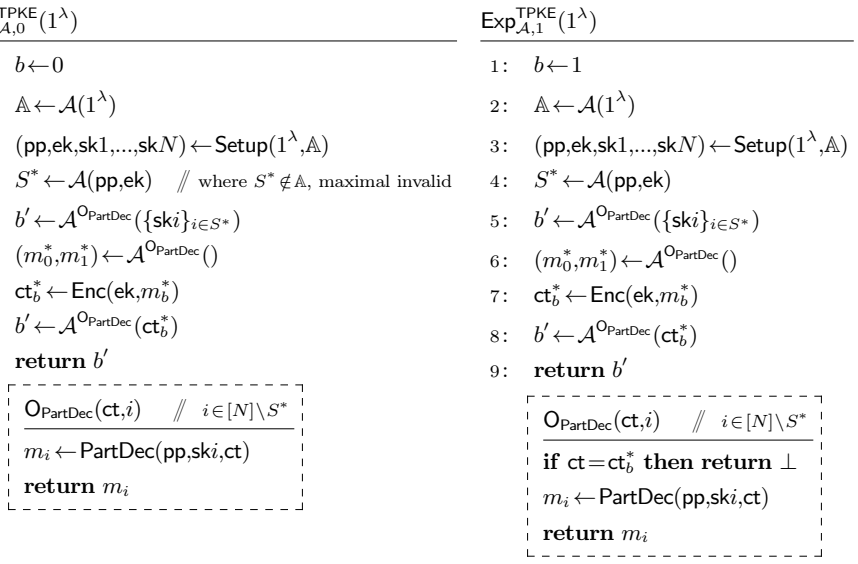

Fig. 1: IND-CCA Security Experiments for Threshold PKE

- $\mu \leftarrow \mathsf{Dec}(sk,ct)$: *On input secret key* $sk=s$ *and ciphertext* $ct=(ct_0,ct_1)$*, computes* $\tilde{\mu}=ct_0-ct_1\cdot s\in R_q$*, and outputs* $\mu=\lfloor \frac{p}{q}\cdot\tilde{\mu}\rceil \bmod p\in R_p$.
- $ct_{add} \leftarrow \mathsf{Add}(ct^{(1)},ct^{(2)})$: *Outputs* $ct_{add}=(ct_0^{(1)}+ct_0^{(2)},ct_1^{(1)}+ct_1^{(2)})$.
- $ct_{mult} \leftarrow \mathsf{Mult}(ct^{(1)},ct^{(2)},\mathsf{rlk})$: *Computes tensor product* $ct_0' = ct_0^{(1)}\cdot ct_0^{(2)}$, $ct_1' = ct_0^{(1)}\cdot ct_1^{(2)} + ct_1^{(1)}\cdot ct_0^{(2)}$, $ct_2' = ct_1^{(1)}\cdot ct_1^{(2)}$, *applies relinearization* $(ct_0,ct_1) = \mathsf{Relin}((ct_0',ct_1',ct_2'),\mathsf{rlk})$*, and outputs* (ct_0,ct_1).
- $\mathsf{rlk} \leftarrow \mathsf{RelinKeyGen}(sk)$: *Generates relinearization key for converting degree-2 to degree-1 ciphertexts.*

Theorem 1 ([118]). *If the* $\mathsf{RLWE}_{n,q,\chi_{key},\chi_{err}}$ *assumption (Definition 3.1) holds, then the* BFV *encryption scheme with parameters* $(n,q,p,\chi_{key},\chi_{err})$ *(Definition 8) is IND-CPA secure; i.e., for any PPT adversary* $\mathcal{A}$ *attacking the IND-CPA security of* BFV*, there exists a PPT algorithm* $\mathcal{B}$ *solving* $\mathsf{RLWE}_{n,q,\chi_{key},\chi_{err}}$ *such that* $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{BFV},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{RLWE}}_{n,q,\chi_{key},\chi_{err},\mathcal{B}}(\lambda)+\mathsf{negl}(\lambda)$*, where* $\mathsf{negl}(\lambda)$ *is a negligible function in* λ.

Definition 9 ([119, 120]). *The* `Kyber512` *key encapsulation mechanism based on* MLWE *with parameters* $(n=256,k=2,q=3329,\eta_1=3,\eta_2=2)$ *where* $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$*, provides* $\mathsf{Kyber}=(\mathsf{Gen},\mathsf{Encaps},\mathsf{Decaps})$.

- $(\mathsf{ek},\mathsf{dk}) \leftarrow \mathsf{Gen}(1^\lambda)$: *Generates seed* $\rho \leftarrow \{0,1\}^{256}$*, expands to matrix* $\mathbf{A}\in R_q^{2\times 2}$*, samples* $\mathbf{s},\mathbf{e} \leftarrow \beta_{\eta_1}^2$ *from centered binomial distribution, computes* $\mathbf{t}=\mathbf{A}\mathbf{s}+\mathbf{e}$*, outputs* $\mathsf{ek}=(\rho,\mathbf{t})$ *and* $\mathsf{dk}=\mathbf{s}$.
- $(ct,K) \leftarrow \mathsf{Encaps}(\mathsf{ek})$: *Samples randomness* $r \leftarrow \{0,1\}^{256}$*, generates shared secret by a key derivation function* $K=\mathsf{KDF}(r)\in\{0,1\}^{256}$ *and ciphertext* ct *encapsulating* r.

– $K \leftarrow \mathsf{Decaps}(\mathsf{dk},ct)$*: Recovers* r *using* dk*, outputs* $K = \mathsf{KDF}(r)$.

Combined with AES-256-CTR*, this provides public-key encryption:* $\mathsf{PKE.Enc}(\mathsf{ek},m) = (ct, \mathsf{AES}_K(m))$ *where* $(ct,K) \leftarrow \mathsf{Encaps}(\mathsf{ek})$ *and* $\mathsf{PKE.Dec}(\mathsf{dk},(ct,c)) = \mathsf{AES}^{-1}_{\mathsf{Decaps}(\mathsf{dk},ct)}(c)$.

Theorem 2 ([121]). *If the* $\mathsf{MLWE}_{n,k,q,\chi}$ *(Definition 3.1) holds, then the* Kyber512 *encryption with* $n=256, k=2, q=3329, \eta_1=3, \eta_2=2$ *(Definition 9) is IND-CCA2-secure in the quantum random oracle model (QROM); i.e., for any quantum polynomial time (QPT) adversary* $\mathcal{A}$ *attacking the IND-CCA2 security of* Kyber*, there exists a QPT algorithm* $\mathcal{B}$ *solving* $\mathsf{MLWE}_{n,k,q,\chi}$ *such that* $\mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathsf{Kyber},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{MLWE}}_{n,k,q,\chi,\mathcal{B}}(\lambda) + \mathsf{negl}(\lambda)$.

Definition 10 ([122]). *The Group Action Hashed Diffie-Hellman (*GAHDH*) is a Non-Interactive Key Exchange (*NIKE*) based on Commutative Supersingular Isogeny Diffie-Hellman (*CSIDH*) group action with set* $\mathcal{E}$ *of supersingular elliptic curves over* $\mathbb{F}_p$ *and commutative group* $G = \mathsf{cl}(\mathcal{O})$ *acting freely and transitively on* $\mathcal{E}$*, provides* $\mathsf{GAHDH} = (\mathsf{Gen},\mathsf{SharedKey})$.

– $(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda,\mathsf{id})$*: Samples* $\mathsf{sk} = [\mathbf{e}] \leftarrow G$ *where* $\mathbf{e} \in \{-B,...,B\}^{74}$ *for bound* $B = 5$ *(*CSIDH-512*), computes public key* $\mathsf{pk} = \mathsf{sk} \star E_0$ *where* $E_0 \in \mathcal{E}$ *is base curve with known endomorphism ring, outputs* $(\mathsf{pk},\mathsf{sk})$.
– $K \leftarrow \mathsf{SharedKey}(\mathsf{pk}_1,\mathsf{id}_1,\mathsf{sk}_2,\mathsf{id}_2)$*: Computes shared curve* $E_{shared} = \mathsf{sk}_2 \star \mathsf{pk}_1$*, outputs* $K = H(\mathsf{id}_1 \| \mathsf{id}_2 \| j(E_{shared}))$ *where* $j : \mathcal{E} \to \mathbb{F}_p$ *is* j*-invariant and* H *is hash function.*

Theorem 3 ([123]). *If the* GA-StCDH *with quantum* DDH *oracle access (Definition 3.1) holds in the* CSIDH *group action, then the* GAHDH *protocol (Definition 10) is one-pair malicious-key secure in the QROM; i.e., for any QPT adversary* $\mathcal{A}$ *attacking the one-pair malicious-key security of* GAHDH*, there exists a QPT algorithm* $\mathcal{B}$ *solving* GA-StCDH *such that* $\mathsf{Adv}^{\mathsf{1p\text{-}mk}}_{\mathsf{GAHDH},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathsf{GA\text{-}StCDH}}_{\mathcal{B}}(\lambda) + \mathsf{negl}(\lambda)$.

Definition 11 ([124]). *The* PTRHash *construction, an optimized FCH-based minimal perfect hash function, provides static dictionary* $\mathsf{DS} = (\mathsf{Insert},\mathsf{Get})$ *with key space* $\{0,1\}^k$*, value space* $\{0,1\}^v$*, randomness space* $\mathcal{R} = \{0,1\}^{128}$*, and dictionary space* $\mathcal{D}$.

– $D \leftarrow \mathsf{Insert}(S,R)$*: On input set* $S = \{(x_1,y_1),...,(x_m,y_m)\} \subseteq \{0,1\}^k \times \{0,1\}^v$ *with distinct keys and seed* $R \in \mathcal{R}$*, constructs* PTRHash *data structure:*
  1. *Uses seed* R *to derive hash functions* $h_0,h_1,h_2 : \{0,1\}^k \to [0,\alpha m]$ *for* $\alpha = 1.01$
  2. *Constructs 3-regular hypergraph* $H = (V,E)$ *with* $|V| = \alpha m$ *and hyperedges* $\{h_0(x_i),h_1(x_i),h_2(x_i)\}$ *for each* x_i
  3. *Computes orientation (peeling order) of* H *and stores displacement values*
  4. *Allocates array* $T[0...\alpha m - 1]$ *and places each* y_i *at position determined by hash and displacement*

  Outputs $D = (R,T)$ *where* $|D| \approx m \cdot v + \frac{m}{3}$ *bytes.*
– $y \leftarrow \mathsf{Get}(D,R,x)$*: Parses* $D = (R,T)$*, computes hash values* $h_0(x),h_1(x),h_2(x)$ *using seed* R*, retrieves displacement to compute position* pos*, outputs* $y = T[\mathsf{pos}]$ *if* x *was in original set* S*, else outputs* $\perp$.

The correctness of PTRHash [124] is derived from the fact that for all $(x,y) \in S$, $\Pr[\mathsf{Get}(\mathsf{Insert}(S,R),R,x) = y] \geq 1 - 2^{-128}$.

We construct an instantiation of the exact lattice-based proof system introduced by Esgin et al. [125] [1], which is based on the MSIS for computational soundness and the MLWE for computational zero-knowledge

Definition 12 ([125]). *Built upon the system from [125], the proof system for proving $v \in [0,2^\ell - 1]$ with parameters $(n,q,\ell,\kappa,\lambda,k,d,\chi)$ where d is power of two, $q \equiv 1 \pmod{2d}$ fully splits, $k \mid d$ is repetition rate, $R_q = \mathbb{Z}_q[X]/(X^d+1)$, κ and λ are module ranks for* MSIS *and* MLWE*, consists of* $\mathsf{RangeProof} = (\mathsf{Setup},\mathsf{Commit},\mathsf{Prove},\mathsf{Verify})$.

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \ell)$*: On input security parameter 1^λ and bit-length ℓ, samples commitment matrix $\mathbf{B}_0 \leftarrow\$ R_q^{\kappa \times (\lambda+\kappa+\lceil \ell/d \rceil+3)}$ and vectors $\mathbf{b}_1, \ldots, \mathbf{b}_{\lceil \ell/d \rceil+3} \leftarrow\$ R_q^{\lambda+\kappa+\lceil \ell/d \rceil+3}$, outputs public parameters $\mathsf{pp} = (\mathbf{B}_0, \{\mathbf{b}_i\}_{i=1}^{\lceil \ell/d \rceil+3})$.*
- $(C,\mathbf{r}) \leftarrow \mathsf{Commit}(\mathsf{pp},v)$*: Takes input public parameters* pp *and value $v \in [0,2^\ell-1]$, computes binary decomposition $v = \sum_{i=0}^{\ell-1} b_i \cdot 2^i$ where $b_i \in \{0,1\}$, encodes bits as polynomials $\hat{b}_1,\ldots,\hat{b}_{\lceil \ell/d \rceil} \in R_q$ where $\mathsf{NTT}(\hat{b}_j) = (b_{(j-1)d},\ldots,b_{\min(jd,\ell)-1},0,\ldots,0)^T \in \mathbb{Z}_q^d$, samples randomness $\mathbf{r} \leftarrow\$ \chi^{(\lambda+\kappa+\lceil \ell/d \rceil+3)d}$, computes commitment components $\mathbf{t}_0 = \mathbf{B}_0\mathbf{r} \in R_q^\kappa$ and $t_j = \langle \mathbf{b}_j, \mathbf{r} \rangle + \hat{b}_j \in R_q$ for $j = 1,\ldots,\lceil \ell/d \rceil$, outputs commitment $C = (\mathbf{t}_0, t_1, \ldots, t_{\lceil \ell/d \rceil})$ and opening $\mathbf{r}$.*
- $\pi \leftarrow \mathsf{Prove}(\mathsf{pp},v,C,\mathbf{r})$*: On input public parameters* pp*, value v, commitment C, and randomness $\mathbf{r}$, the proving algorithm executes:*
  1. *For each bit polynomial $\hat{b}_j$, proves $\hat{b}_j(\hat{b}_j - 1) = 0$ using product proof from [125] with automorphism optimization, which shows $\mathsf{NTT}(\hat{b}_j) \in \{0,1\}^d$ coefficient-wise (binary proof).*
  2. *Constructs coefficient vector $\mathbf{s} = \mathsf{NTT}(\hat{b}_1) \| \cdots \| \mathsf{NTT}(\hat{b}_{\lceil \ell/d \rceil}) \in \mathbb{Z}_q^{\lceil \ell/d \rceil \cdot d}$, defines weight vector $\mathbf{w} = (2^0, 2^1, \ldots, 2^{\ell-1}, 0, \ldots, 0)^T \in \mathbb{Z}_q^{\lceil \ell/d \rceil \cdot d}$, proves linear relation $\langle \mathbf{w}, \mathbf{s} \rangle = v \bmod q$ using unstructured linear proof with soundness amplification via automorphisms (linear proof).*
  3. *Proves knowledge of opening $\mathbf{r}$ for commitment C using automorphism opening proof with k parallel repetitions and challenges $\sigma^i(c)$ for $i = 0,\ldots,k-1$ (opening proof).*

  Combines all proofs using Fiat-Shamir transform with challenge $c \leftarrow\$ \mathcal{C} = \{-1,0,1\}^d$, outputs proof $\pi = (\mathbf{t}_0, \{t_j\}_{j=1}^{\lceil \ell/d \rceil}, \{t_{\lceil \ell/d \rceil + j}\}_{j=1}^{3}, h, \{\mathbf{z}_i\}_{i=0}^{k-1})$ where the values $t_{\lceil \ell/d \rceil+1}, t_{\lceil \ell/d \rceil+2}, t_{\lceil \ell/d \rceil+3}$ are garbage term commitments, $h \in R_q$ is masked linear combination with $h_0 = \cdots = h_{k-1} = 0$, and $\mathbf{z}_i \in R_q^{\lambda+\kappa+\lceil \ell/d \rceil+3}$ are masked openings with $\|\mathbf{z}_i\|_\infty < \delta_1 - T$.
- $\{0,1\} \leftarrow \mathsf{Verify}(\mathsf{pp},v,C,\pi)$*: Takes input public parameters* pp*, claimed value v, commitment C, and proof π, the verification algorithm parses the content of $\pi = (\mathbf{t}_0, \{t_j\}, \{t_{\lceil \ell/d \rceil + j}\}, h, \{\mathbf{z}_i\})$, recomputes challenge c via Fiat-Shamir, verifies for all $i = 0,\ldots,k-1$:*
  1. *$\|\mathbf{z}_i\|_\infty \overset{?}{<} \delta_1 - T$*

[1] We refer to a proof system that exactly proves knowledge of a ternary secret vector, and hence does not have any knowledge gap, as an *exact proof system*.

2. $\mathbf{B}_0\mathbf{z}_i \stackrel{?}{=} \mathbf{w}_i + \sigma^i(c)\mathbf{t}_0$
3. $\sum_{i=0}^{k-1}\sum_{j=1}^{\lceil \ell/d\rceil}\alpha_{i\lceil \ell/d\rceil+j}\sigma^{-i}(\mathbf{f}_j^{(i)}(\mathbf{f}_j^{(i)}-\sigma^i(c))(\mathbf{f}_j^{(i)}+\sigma^i(c)))+\mathbf{f}_{\lceil \ell/d\rceil+2}+c\mathbf{f}_{\lceil \ell/d\rceil+3} \stackrel{?}{=} v$ *where* $\mathbf{f}_j^{(i)} = \langle \mathbf{b}_j, \mathbf{z}_i\rangle - \sigma^i(c)t_j$ *and* $\{\alpha_i\}$ *are random challenge polynomials (product proof verification)*
4. $h_0 \stackrel{?}{=} \cdots \stackrel{?}{=} h_{k-1} \stackrel{?}{=} 0$
5. *For each* i, *verifies*

$$\sum_{\mu=0}^{k-1}\frac{1}{k}X^{\mu}\sum_{\nu=0}^{k-1}\sum_{j=1}^{\lceil \ell/d\rceil}\sigma^{\nu}(\langle d\psi_j^{(\mu)}\mathbf{b}_j, \mathbf{z}_{i-\nu \bmod k}\rangle) + \langle \mathbf{b}_{\lceil \ell/d\rceil+1}, \mathbf{z}_i\rangle \stackrel{?}{=} v_i' + \sigma^i(c)(\tau + t_{\lceil \ell/d\rceil+1} - h)$$

where τ *is commitment to linear combination and* $\psi_j^{(\mu)}$ *encode weight vector* $\mathbf{w}$ *(linear proof verification)*

Outputs 1 *if all checks pass, else* 0.

RangeProof achieves computational soundness under MSIS with soundness error $(3p)^k$ where p is the maximum probability of challenge coefficients modulo $X^k - \zeta^k$ per Lemma 2.2 in [125], and computational zero-knowledge under MLWE via simulation with rejection sampling at rate $\approx 1/e$ per repetition.

4 Homomorphic Silent-setup Threshold Cryptosystem

Definition 13. *A Homomorphic Silent Threshold Encryption scheme (*HSTE*) with parameters* (n,q,p,m,N) *where* $m = poly(\lambda)$ *is committee size,* N *is total number of parties, and corruption bound* $t \leq (1/2-\epsilon)\cdot N$ *for constant* $\epsilon > 0$, *consists of* HSTE = (Gen,Enc,PartDec,PartVerify,Dec,HomAdd,HomMult) *built upon* BFV, Kyber, GAHDH, PTRHash, *and random oracle* H.

- $(pk,sk) \leftarrow \mathsf{Gen}(1^\lambda,\mathsf{id})$*: On input security parameter* 1^λ *and identity* id, *the key generation algorithm generates* BFV *keys* $(pk_{bfv}, sk_{bfv}, \mathsf{rlk}) \leftarrow \mathsf{BFV.KeyGen}(1^\lambda)$ *where* $pk_{bfv} = (\mathbf{a},\mathbf{b})$, $sk_{bfv} = s$, *then generates* NIKE *keys* $(\mathsf{pk}_{nike}, \mathsf{sk}_{nike}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda,\mathsf{id})$, *next generate* PKE *keys* $(\mathsf{ek}_{kyber},\mathsf{dk}_{kyber}) \leftarrow \mathsf{Kyber.Gen}(1^\lambda)$, *and finally outputs* $pk = (pk_{bfv},\mathsf{pk}_{nike},\mathsf{ek}_{kyber},\mathsf{rlk})$ *and* $sk = (sk_{bfv},\mathsf{sk}_{nike},\mathsf{dk}_{kyber})$.
- $ct \leftarrow \mathsf{Enc}(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu)$*: Gets the inputs public keys with identities and message* $\mu \in R_p$, *the encryption algorithm samples random committee* $\mathcal{C} \leftarrow \binom{[N]}{m}$, *then parses committee members'* BFV *public keys by extracting* $pk_{bfv,i} = (\mathbf{a}_i,\mathbf{b}_i)$ *for each* $i \in \mathcal{C}$, *and computes aggregated* BFV *public key* $pk_{bfv,\mathcal{C}} = \left(\mathbf{a}_0, \frac{1}{m}\sum_{i\in\mathcal{C}}\mathbf{b}_i\right)$ *where* $\mathbf{a}_0 = \mathbf{a}_i$ *for any* $i \in \mathcal{C}$ *(all parties use same* $\mathbf{a}$*). It encrypts the message using* BFV *as* $(ct_0,ct_1) \leftarrow \mathsf{BFV.Enc}(pk_{bfv,\mathcal{C}},\mu)$, *generates ephemeral* NIKE *key* $(\mathsf{pk}_{eph},\mathsf{sk}_{eph}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda,0)$, *and then for each party* $j \in \mathcal{C}$, *computes shared* NIKE *key* $K_j \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{nike,j},\mathsf{id}_j,\mathsf{sk}_{eph},0)$, *samples randomness* $r_j \leftarrow \{0,1\}^\lambda$, *computes tag* $h_j = H(j\|r_j) \in \{0,1\}^{256}$, *encrypts randomness using* Kyber *as* $(ct_{kyber,j},K_{kem,j}) \leftarrow \mathsf{Kyber.Encaps}(\mathsf{ek}_{kyber,j})$ *followed by* $c_j = \mathsf{AES}_{K_{kem,j}}(r_j)$, *and sets dictionary value* $v_j = (ct_{kyber,j},c_j,h_j)$. *The algorithm constructs a static dictionary by setting* $S = \{(K_j,v_j) : j \in \mathcal{C}\}$, *sampling dictionary seed* $R \leftarrow \{0,1\}^{128}$, *and building the dictionary* $\mathcal{D} \leftarrow \mathsf{DS.Insert}(S,R)$. *It outputs ciphertext* $ct = (ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{eph})$.

- $\sigma_i \leftarrow \mathsf{PartDec}(sk_i, \mathsf{id}_i, ct)$: *Receives input secret key* $sk_i = (s_i, \mathsf{sk}_{nike,i}, \mathsf{dk}_{kyber,i})$, *identity* id_i, *and ciphertext* $ct = (ct_0, ct_1, \mathcal{D}, R, \mathsf{pk}_{eph})$, *the partial decryption algorithm computes shared* NIKE *key* $K_i \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{eph}, 0, \mathsf{sk}_{nike,i}, \mathsf{id}_i)$, *retrieves from dictionary* $v_i = (ct_{kyber,i}, c_i, h_i) \leftarrow \mathsf{DS.Get}(\mathcal{D}, R, K_i)$ *and outputs* $\perp$ *if* $v_i = \perp$, *decapsulates* Kyber *as* $K_{kem,i} \leftarrow \mathsf{Kyber.Decaps}(\mathsf{dk}_{kyber,i}, ct_{kyber,i})$, *decrypts randomness* $r_i = \mathsf{AES}^{-1}_{K_{kem,i}}(c_i)$, *verifies tag by checking if* $H(i \| r_i) \neq h_i$ *and outputting* $\perp$ *if so, computes* BFV *partial decryption* $d_i = ct_1 \cdot s_i \in R_q$, *and outputs partial decryption* $\sigma_i = (i, d_i, r_i)$.
- $\{0,1\} \leftarrow \mathsf{PartVerify}(pk_i, ct, \sigma_i)$: *After getting public key* pk_i, *ciphertext* $ct = (ct_0, ct_1, \mathcal{D}, R, \mathsf{pk}_{eph})$, *and partial decryption* $\sigma_i = (i, d_i, r_i)$ *as input, the verification algorithm parses* pk_i *to extract* $\mathsf{pk}_{nike,i}$ *and* id_i, *computes shared key* $K_i \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{eph}, 0, \mathsf{sk}_{nike,i}, \mathsf{id}_i)$ *(computable from public information), retrieves tag* $(-, -, h_i) \leftarrow \mathsf{DS.Get}(\mathcal{D}, R, K_i)$, *verifies tag by checking* $H(i \| r_i) \stackrel{?}{=} h_i$, *and outputs* 1 *if verification passes, else* 0.
- $\mu^* \leftarrow \mathsf{Dec}(ct, \{\sigma_i\}_{i \in \mathcal{S}})$: *On input ciphertext* $ct = (ct_0, ct_1, \mathcal{D}, R, \mathsf{pk}_{eph})$ *and partial decryptions* $\{\sigma_i = (i, d_i, r_i)\}_{i \in \mathcal{S}}$, *the decryption algorithm verifies and filters as* $\mathcal{S}_v = \{i \in \mathcal{S} : \mathsf{PartVerify}(pk_i, ct, \sigma_i) = 1\}$, *checks threshold by outputting* $\perp$ *if* $|\mathcal{S}_v| < m/2$, *computes Lagrange coefficients over* $\mathbb{Z}_q$ *for each* $i \in \mathcal{S}_v$ *as* $\lambda_i = \prod_{j \in \mathcal{S}_v \setminus \{i\}} \frac{j}{j-i} \bmod q$, *aggregates partial decryptions as* $d_{agg} = \frac{1}{m} \sum_{i \in \mathcal{S}_v} \lambda_i \cdot d_i \in R_q$, *completes* BFV *decryption as* $\mu^* = \left\lfloor \frac{p}{q} \cdot (ct_0 - d_{agg}) \right\rceil \bmod p \in R_p$, *and outputs plaintext* $\mu^* \in R_p$.
- $ct_{add} \leftarrow \mathsf{HomAdd}(ct^{(1)}, ct^{(2)})$: *Accepts ciphertexts* $ct^{(1)} = (ct_0^{(1)}, ct_1^{(1)}, \mathcal{D}, R, \mathsf{pk}_{eph})$ *and* $ct^{(2)} = (ct_0^{(2)}, ct_1^{(2)}, \mathcal{D}, R, \mathsf{pk}_{eph})$ *with identical committee structure as input, the homomorphic addition algorithm verifies matching committees by checking* $\mathcal{D}^{(1)} = \mathcal{D}^{(2)}$, $R^{(1)} = R^{(2)}$, $\mathsf{pk}_{eph}^{(1)} = \mathsf{pk}_{eph}^{(2)}$, *computes* BFV *addition* $(ct_{0,add}, ct_{1,add}) \leftarrow \mathsf{BFV.Add}((ct_0^{(1)}, ct_1^{(1)}), (ct_0^{(2)}, ct_1^{(2)}))$, *and outputs* $ct_{add} = (ct_{0,add}, ct_{1,add}, \mathcal{D}, R, \mathsf{pk}_{eph})$.
- $ct_{mult} \leftarrow \mathsf{HomMult}(ct^{(1)}, ct^{(2)})$: *On input ciphertexts with matching committee structure, the homomorphic multiplication algorithm verifies matching committees as in* HomAdd, *extracts aggregated relinearization key from committee as* $\mathsf{rlk}_{\mathcal{C}} = \frac{1}{m} \sum_{i \in \mathcal{C}} \mathsf{rlk}_i$ *where* rlk_i *is extracted from* pk_i, *computes* BFV *multiplication* $(ct_{0,mult}, ct_{1,mult}) \leftarrow \mathsf{BFV.Mult}((ct_0^{(1)}, ct_1^{(1)}), (ct_0^{(2)}, ct_1^{(2)}), \mathsf{rlk}_{\mathcal{C}})$, *and outputs* $ct_{mult} = (ct_{0,mult}, ct_{1,mult}, \mathcal{D}, R, \mathsf{pk}_{eph})$.

4.1 Analyzing Properties of HSTE as a TPKE

Both HSTE and Garg et al. [126] achieve similar corruption thresholds just below $n/2$, but HSTE supports homomorphic operations (HomAdd, HomMult) on encrypted data with matching committees, enabling computation on ciphertexts without decryption; moreover, HSTE is post-quantum due to its lattice-based structure, but Garg et al. [126] is vulnerable to quantum attacks.

Formal proofs for all theorems of this section are provided in Appendix A.

Theorem 4 (Correctness). *Let* HSTE *be the Homomorphic Silent Threshold Encryption (Definition 13) with parameters* (n, q, p, m, N) *where* $m = \mathrm{poly}(\lambda)$ *is the committee size,* N *is the total number of parties, and threshold* $\tau = \lceil m/2 \rceil$. *Assume:*

HSTE.Gen$(1^\lambda,\mathsf{id})$

$(pk_{\mathtt{bfv}},sk_{\mathtt{bfv}},\mathsf{rlk}) \leftarrow \mathsf{BFV.KeyGen}(1^\lambda)$
$pk_{\mathtt{bfv}} \leftarrow (\mathbf{a},\mathbf{b})$
$sk_{\mathtt{bfv}} \leftarrow s$
$(\mathsf{pk}_{\mathtt{nike}},\mathsf{sk}_{\mathtt{nike}}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda,\mathsf{id})$
$(\mathsf{ek}_{\mathtt{kyber}},\mathsf{dk}_{\mathtt{kyber}}) \leftarrow \mathsf{Kyber.Gen}(1^\lambda)$
$pk \leftarrow (pk_{\mathtt{bfv}},\mathsf{pk}_{\mathtt{nike}},\mathsf{ek}_{\mathtt{kyber}},\mathsf{rlk})$
$sk \leftarrow (sk_{\mathtt{bfv}},\mathsf{sk}_{\mathtt{nike}},\mathsf{dk}_{\mathtt{kyber}})$
return (pk,sk)

HSTE.Enc$(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu)$

$\mathcal{C} \leftarrow\!\!\$ \binom{[N]}{m}$
for $i \in \mathcal{C}$ **do**
  $pk_{\mathtt{bfv},i} \leftarrow (\mathbf{a}_i,\mathbf{b}_i)$
$\forall i \in \mathcal{C}: \mathbf{a}_0 \leftarrow \mathbf{a}_i$
$pk_{\mathtt{bfv},\mathcal{C}} \leftarrow \left(\mathbf{a}_0, \frac{1}{m}\sum_{i\in\mathcal{C}} \mathbf{b}_i\right)$
$(ct_0,ct_1) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}},\mu)$
$(\mathsf{pk}_{\mathtt{eph}},\mathsf{sk}_{\mathtt{eph}}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda,0)$
$S \leftarrow \emptyset$
for $j \in \mathcal{C}$ **do**
  $K_j \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{nike},j},\mathsf{id}_j,\mathsf{sk}_{\mathtt{eph}},0)$
  $r_j \leftarrow\!\!\$ \{0,1\}^\lambda$
  $h_j \leftarrow H(j\|r_j) \in \{0,1\}^{256}$
  $(ct_{\mathtt{kyber},j},K_{\mathtt{kem},j}) \leftarrow \mathsf{Kyber.Encaps}(\mathsf{ek}_{\mathtt{kyber},j})$
  $c_j \leftarrow \mathsf{AES}_{K_{\mathtt{kem},j}}(r_j)$
  $v_j \leftarrow (ct_{\mathtt{kyber},j},c_j,h_j)$
  $S \leftarrow S \cup \{(K_j,v_j)\}$
$R \leftarrow\!\!\$ \{0,1\}^{128}$
$\mathcal{D} \leftarrow \mathsf{DS.Insert}(S,R)$
$ct \leftarrow (ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathtt{eph}})$
return ct

HSTE.PartVerify(pk_i,ct,σ_i)

$(ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathtt{eph}}) \leftarrow ct$
$(i,d_i,r_i) \leftarrow \sigma_i$
$(\mathsf{pk}_{\mathtt{nike},i},\mathsf{id}_i) \leftarrow \mathsf{ParsePK}(pk_i)$
$K_i \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{eph}},0,\mathsf{sk}_{\mathtt{nike},i},\mathsf{id}_i)$
$(-,-,h_i) \leftarrow \mathsf{DS.Get}(\mathcal{D},R,K_i)$
if $H(i\|r_i) = h_i$ **then**
  return 1
else
  return 0

HSTE.PartDec(sk_i,id_i,ct)

$(s_i,\mathsf{sk}_{\mathtt{nike},i},\mathsf{dk}_{\mathtt{kyber},i}) \leftarrow sk_i$
$(ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathtt{eph}}) \leftarrow ct$
$K_i \leftarrow \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{eph}},0,\mathsf{sk}_{\mathtt{nike},i},\mathsf{id}_i)$
$v_i \leftarrow \mathsf{DS.Get}(\mathcal{D},R,K_i)$
if $v_i = \perp$ **then return** $\perp$
$(ct_{\mathtt{kyber},i},c_i,h_i) \leftarrow v_i$
$K_{\mathtt{kem},i} \leftarrow \mathsf{Kyber.Decaps}(\mathsf{dk}_{\mathtt{kyber},i},ct_{\mathtt{kyber},i})$
$r_i \leftarrow \mathsf{AES}^{-1}_{K_{\mathtt{kem},i}}(c_i)$
if $H(i\|r_i) \neq h_i$ **then return** $\perp$
$d_i \leftarrow ct_1 \cdot s_i \in R_q$
$\sigma_i \leftarrow (i,d_i,r_i)$
return σ_i

HSTE.Dec$(ct,\{\sigma_i\}_{i\in\mathcal{S}})$

$(ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathtt{eph}}) \leftarrow ct$
$\mathcal{S}_v \leftarrow \{i \in \mathcal{S} : \mathsf{HSTE.PartVerify}(pk_i,ct,\sigma_i) = 1\}$
if $|\mathcal{S}_v| < m/2$ **then return** $\perp$
for $i \in \mathcal{S}_v$ **do**
  $\lambda_i \leftarrow \prod_{j\in\mathcal{S}_v\setminus\{i\}} \frac{j}{j-i} \bmod q$
$d_{\mathsf{agg}} \leftarrow \frac{1}{m}\sum_{i\in\mathcal{S}_v} \lambda_i \cdot d_i \in R_q$
$\mu^* \leftarrow \left\lfloor \frac{p}{q} \cdot (ct_0 - d_{\mathsf{agg}}) \right\rceil \bmod p \in R_p$
return μ^*

HSTE.HomAdd$(ct^{(1)},ct^{(2)})$

$(ct_0^{(1)},ct_1^{(1)},\mathcal{D}^{(1)},R^{(1)},\mathsf{pk}_{\mathtt{eph}}^{(1)}) \leftarrow ct^{(1)}$
$(ct_0^{(2)},ct_1^{(2)},\mathcal{D}^{(2)},R^{(2)},\mathsf{pk}_{\mathtt{eph}}^{(2)}) \leftarrow ct^{(2)}$
$\mathsf{require}(\mathcal{D}^{(1)} = \mathcal{D}^{(2)} \wedge R^{(1)} = R^{(2)} \wedge \mathsf{pk}_{\mathtt{eph}}^{(1)} = \mathsf{pk}_{\mathtt{eph}}^{(2)})$
$(ct_{0,\mathtt{add}},ct_{1,\mathtt{add}}) \leftarrow \mathsf{BFV.Add}((ct_0^{(1)},ct_1^{(1)}),(ct_0^{(2)},ct_1^{(2)}))$
$ct_{\mathtt{add}} \leftarrow (ct_{0,\mathtt{add}},ct_{1,\mathtt{add}},\mathcal{D}^{(1)},R^{(1)},\mathsf{pk}_{\mathtt{eph}}^{(1)})$
return ct_{add}

HSTE.HomMult$(ct^{(1)},ct^{(2)})$

$(ct_0^{(1)},ct_1^{(1)},\mathcal{D}^{(1)},R^{(1)},\mathsf{pk}_{\mathtt{eph}}^{(1)}) \leftarrow ct^{(1)}$
$(ct_0^{(2)},ct_1^{(2)},\mathcal{D}^{(2)},R^{(2)},\mathsf{pk}_{\mathtt{eph}}^{(2)}) \leftarrow ct^{(2)}$
$\mathsf{require}(\mathcal{D}^{(1)} = \mathcal{D}^{(2)} \wedge R^{(1)} = R^{(2)} \wedge \mathsf{pk}_{\mathtt{eph}}^{(1)} = \mathsf{pk}_{\mathtt{eph}}^{(2)})$
$\mathsf{rlk}_{\mathcal{C}} \leftarrow \frac{1}{m}\sum_{i\in\mathcal{C}} \mathsf{rlk}_i$
$(ct_{0,\mathtt{mult}},ct_{1,\mathtt{mult}}) \leftarrow \mathsf{BFV.Mult}((ct_0^{(1)},ct_1^{(1)}),(ct_0^{(2)},ct_1^{(2)}),\mathsf{rlk}_{\mathcal{C}})$
$ct_{\mathtt{mult}} \leftarrow (ct_{0,\mathtt{mult}},ct_{1,\mathtt{mult}},\mathcal{D}^{(1)},R^{(1)},\mathsf{pk}_{\mathtt{eph}}^{(1)})$
return $ct_{\mathtt{mult}}$

Fig. 2: Homomorphic Silent-setup Threshold Encryption (HSTE) Scheme from Lattices

1. *The underlying* BFV *scheme satisfies decryption correctness (i.e., noise remains bounded:* $\|\mathbf{e}\|_\infty < q/(2p)$ *with overwhelming probability),*
2. *The* GAHDH NIKE *protocol satisfies correctness (i.e.,* $\Pr[\mathsf{SharedKey}(\mathsf{pk}_A,\mathsf{id}_A,\mathsf{sk}_B,\mathsf{id}_B) = \mathsf{SharedKey}(\mathsf{pk}_B,\mathsf{id}_B,\mathsf{sk}_A,\mathsf{id}_A)] \geq 1-\mathsf{negl}(\lambda)$*),*
3. *The* Kyber *key encapsulation mechanism satisfies* δ*-correctness for* $\delta = \mathsf{negl}(\lambda)$*,*
4. *The* PTRHash *dictionary satisfies lookup correctness with failure probability* 2^{-128}*,*
5. *The random oracle* $H : \{0,1\}^* \to \{0,1\}^{256}$ *is collision-resistant.*

Then HSTE *satisfies decryption correctness (Definition 4) and partial verification correctness (Definition 5), i.e., for all security parameters* $\lambda \in \mathbb{N}$*, all messages* $\mu \in R_p$*, all sets of public keys with identities* $\{(pk_i,\mathsf{id}_i)\}_{i=1}^N$ *generated via* HSTE.Gen*, and all subsets* $\mathcal{S} \subseteq [N]$ *with* $|\mathcal{S}| \geq \tau$*, we have:*

$$\Pr\left[\begin{array}{c} \forall i \in [N] : (pk_i,sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda,\mathsf{id}_i); \\ ct \leftarrow \mathsf{HSTE.Enc}(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu); \\ \mathcal{C} \leftarrow \text{committee in } ct; \quad \mathcal{S}' \leftarrow \mathcal{S} \cap \mathcal{C}, |\mathcal{S}'| \geq \tau; \\ \forall i \in \mathcal{S}' : \sigma_i \leftarrow \mathsf{HSTE.PartDec}(sk_i,\mathsf{id}_i,ct); \\ \mu' \leftarrow \mathsf{HSTE.Dec}(ct,\{\sigma_i\}_{i\in\mathcal{S}'}) : \mu' = \mu \end{array}\right] \geq 1-\mathsf{negl}(\lambda).$$

Theorem 5 (Compactness). *Suppose* HSTE *be the Homomorphic Silent Threshold Encryption scheme (Definition 13) with parameters* (n,q,p,m,N) *where* n *is the ring dimension,* q *is the ciphertext modulus,* p *is the plaintext modulus,* $m = \mathrm{poly}(\lambda)$ *is the committee size, and* N *is the total number of parties. Then* HSTE *satisfies compactness (Definition 3); specifically, there exist polynomials* $\mathsf{poly}_1(\cdot)$ *and* $\mathsf{poly}_2(\cdot)$ *such that for all security parameters* $\lambda \in \mathbb{N}$ *and all messages* $\mu \in R_p$*:*

$$\begin{aligned} |ct| &\leq \mathsf{poly}_1(\lambda), \\ |pk_i| &\leq \mathsf{poly}_2(\lambda) \quad \textit{for all } i \in [N], \end{aligned}$$

where $ct \leftarrow \mathsf{HSTE.Enc}(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu)$ *and* $(pk_i,sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda,\mathsf{id}_i)$*. The ciphertext size* $|ct|$ *is* independent of N *and depends only on the committee size* $m = \mathrm{poly}(\lambda)$*.*

Theorem 6 (Security). *According to semantic security of* BFV *under* RLWE *(Theorem 1), IND-CCA2-security of* Kyber *under* MLWE *(Theorem 2),* GAHDH *is one-pair malicious-key secure under* GA-StCDH *assumption in* CSIDH *(Theorem 3), the Homomorphic Silent Threshold Encryption* HSTE *is IND-CCA-secure (Definition 6).*

Theorem 7 states that the probability that an adversary produces a partial decryption σ^* that passes verification but differs from the honest partial decryption σ_{i^*} is negligible.

Theorem 7 (Robustness). *Let* HSTE *be the Homomorphic Silent Threshold Encryption scheme (Definition 13) with parameters* (n,q,p,m,N)*. Assume:*

1. *The random oracle* $H : \{0,1\}^* \to \{0,1\}^{256}$ *is modeled as a random oracle,*

2. *The* Kyber *satisfies IND-CCA2 security under* MLWE *(Theorem 2),*
3. GAHDH *satisfies one-pair malicious-key security under* GA-StCDH *(Theorem 3),*
4. *The* PTRHash *dictionary provides collision-resistant lookup (i.e., for distinct keys* $K \neq K'$, $\Pr[\mathsf{DS.Get}(\mathcal{D},R,K) = \mathsf{DS.Get}(\mathcal{D},R,K')] \leq 2^{-128}$*).*

Then HSTE *satisfies robustness (Definition 7); specifically, for any probabilistic polynomial-time adversary* $\mathcal{A}$ *and all security parameters* $\lambda \in \mathbb{N}$*:*

$$\Pr\left[\begin{array}{c} \forall i \in [N] : (pk_i, sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda, \mathsf{id}_i); ct \leftarrow \mathsf{HSTE.Enc}(\{(pk_i, \mathsf{id}_i)\}_{i=1}^N, \mu); \\ (i^*, \sigma^*) \leftarrow \mathcal{A}(ct, \{pk_i, sk_i\}_{i \in [N]}); \sigma_{i^*} \leftarrow \mathsf{HSTE.PartDec}(sk_{i^*}, \mathsf{id}_{i^*}, ct); \\ b_1 \leftarrow \mathsf{HSTE.PartVerify}(pk_{i^*}, ct, \sigma^*); b_2 \leftarrow (\sigma^* \neq \sigma_{i^*}) : b_1 \wedge b_2 = 1 \end{array}\right] \leq \mathsf{negl}(\lambda).$$

5 Post-quantum ElectionGuard with Silent-setup

Definition 14. *The Post-Quantum ElectionGuard (*PQEG*) with silent setup, based on* HSTE *and lattice-based range proofs, with parameters* $(n,q,p,m,N,\ell,\kappa,\lambda_{rp},k,d)$ *where* N *is the number of guardians,* $m = poly(\lambda)$ *is committee size,* ℓ *is bit-length for range proofs, and election manifest* $\mathcal{M}$ *specifies contests with selection limits, consists of* $\mathsf{PQEG} = (\mathsf{Setup}, \mathsf{Vote}, \mathsf{Valid}, \mathsf{Aggregate}, \mathsf{PartTally}, \mathsf{Tally}, \mathsf{Verify})$.

- $(\{(\mathsf{pk}_i, \mathsf{id}_i)\}_{i=1}^N, \{\mathsf{sk}_i\}_{i=1}^N, \mathsf{pp}_{rp}, H_E) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{M}, N)$*: On input security parameter* 1^λ*, election manifest* $\mathcal{M}$*, and number of guardians* N*, the setup algorithm performs independent key generation for each guardian* G_i *as* $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda, i)$ *where* $\mathsf{pk}_i = (pk_{bfv,i}, \mathsf{pk}_{nike,i}, \mathsf{ek}_{kyber,i}, \mathsf{rlk}_i)$ *and* $\mathsf{sk}_i = (s_i, \mathsf{sk}_{nike,i}, \mathsf{dk}_{kyber,i})$*, computes joint election public key* $\mathsf{PK} = \{(\mathsf{pk}_i, i)\}_{i=1}^N$*, generates range proof parameters* $\mathsf{pp}_{rp} \leftarrow \mathsf{RangeProof.Setup}(1^\lambda, \ell)$ *where* $\ell = \lceil \log_2(\max_{contests} L) \rceil$ *for maximum selection limit* L*, computes manifest hash* $H_M = H(\mathcal{M})$*, computes extended base hash* $H_E = H(H_M \| \mathsf{PK} \| \mathsf{pp}_{rp})$*, and outputs public parameters* $(\{(\mathsf{pk}_i, i)\}_{i=1}^N, \mathsf{pp}_{rp}, H_E)$ *and secret keys* $\{\mathsf{sk}_i\}_{i=1}^N$.
- $(B, H_B, \{\pi_j\}_j) \leftarrow \mathsf{Vote}(\mathsf{PK}, \mathcal{M}, \mathsf{pp}_{rp}, \mathbf{v})$*: Takes input election public key* $\mathsf{PK} = \{(\mathsf{pk}_i, i)\}_{i=1}^N$*, manifest* $\mathcal{M}$*, range proof parameters* pp_{rp}*, and voter selections* $\mathbf{v} = \{v_{j,\ell}\}$ *where* $v_{j,\ell} \in \{0,1\}$ *indicates selection* ℓ *in contest* j *(or* $v_{j,\ell} \in [0, R_\ell]$ *for cardinal voting with option limit* R_ℓ*), the voting algorithm samples ballot nonce* $\xi_B \leftarrow \{0,1\}^\lambda$*, and for each contest* j *with options* $\{1,...,n_j\}$ *and selection limit* L_j*:*
  1. *For each option* $\ell \in [n_j]$*, derives selection nonce* $\xi_{j,\ell} = H(H_E \| \xi_B \| j \| \ell)$*, encrypts it as* $ct_{j,\ell} = (ct_{0,j,\ell}, ct_{1,j,\ell}, \mathcal{D}_B, R_B, \mathsf{pk}_{eph,B}) \leftarrow \mathsf{HSTE.Enc}(\mathsf{PK}, v_{j,\ell}; \xi_{j,\ell})$ *using deterministic randomness derived from* $\xi_{j,\ell}$
  2. *Computes commitment* $(C_{j,\ell}, \mathbf{r}_{j,\ell}) \leftarrow \mathsf{RangeProof.Commit}(\mathsf{pp}_{rp}, v_{j,\ell})$
  3. *Generates option correctness proof* $\pi_{j,\ell}^{opt} \leftarrow \mathsf{RangeProof.Prove}(\mathsf{pp}_{rp}, v_{j,\ell}, C_{j,\ell}, \mathbf{r}_{j,\ell})$ *proving* $v_{j,\ell} \in [0, R_\ell]$ *where* $R_\ell = 1$ *for standard voting or* $R_\ell > 1$ *for cardinal methods*

  For each contest j*, computes aggregate selection* $ct_j^{sum} = \mathsf{HSTE.HomAdd}(ct_{j,1}, ..., ct_{j,n_j})$ *encrypting* $\sum_{\ell=1}^{n_j} v_{j,\ell}$*, computes aggregate the value of commitment* $(C_j^{sum}, \mathbf{r}_j^{sum}) \leftarrow \mathsf{RangeProof.Commit}(\mathsf{pp}_{rp}, \sum_\ell v_{j,\ell})$ *where* $\mathbf{r}_j^{sum} = \sum_\ell \mathbf{r}_{j,\ell}$*, generates contest limit*

proof $\pi_j^{\mathit{lim}} \leftarrow \mathsf{RangeProof.Prove}(\mathsf{pp}_{\mathit{rp}}, \sum_\ell v_{j,\ell}, C_j^{\mathit{sum}}, \mathbf{r}_j^{\mathit{sum}})$ *proving* $\sum_{\ell=1}^{n_j} v_{j,\ell} \in [0, L_j]$, *computes contest hash* $\chi_j = H(H_E \| j \| \{ct_{j,\ell}\}_{\ell=1}^{n_j})$, *and constructs ballot* $B = \{\{ct_{j,\ell}\}_{\ell=1}^{n_j}\}_{j=1}^{|\mathcal{M}|}$ *with all selection encryptions, computes ballot confirmation code* $H_B = H(H_E \| \chi_1 \| \cdots \| \chi_{|\mathcal{M}|})$, *and outputs ballot* B, *confirmation code* H_B, *and proofs* $\{\pi_j = (\{\pi_{j,\ell}^{\mathit{opt}}\}_{\ell=1}^{n_j}, \pi_j^{\mathit{lim}})\}_{j=1}^{|\mathcal{M}|}$.

- $\{0,1\} \leftarrow \mathsf{Valid}(\mathsf{PK}, \mathcal{M}, \mathsf{pp}_{\mathit{rp}}, B, \{\pi_j\}_j, H_B)$*: On input election public key* PK, *manifest* $\mathcal{M}$, *range proof parameters* $\mathsf{pp}_{\mathit{rp}}$, *ballot* $B = \{\{ct_{j,\ell}\}_\ell\}_j$, *proofs* $\{\pi_j\}_j$, *and confirmation code* H_B, *the validation algorithm performs:*
  1. *Verifies ballot structure matches manifest* $\mathcal{M}$
  2. *For each contest* j *with selection limit* L_j*:*
     - *For each option* $\ell \in [n_j]$, *parses commitment from* $\pi_{j,\ell}^{\mathit{opt}}$, *verifies option proof* $\mathsf{RangeProof.Verify}(\mathsf{pp}_{\mathit{rp}}, -, C_{j,\ell}, \pi_{j,\ell}^{\mathit{opt}}) \stackrel{?}{=} 1$ *proving encrypted value is in valid range* $[0, R_\ell]$
     - *Verifies contest limit proof* $\mathsf{RangeProof.Verify}(\mathsf{pp}_{\mathit{rp}}, -, C_j^{\mathit{sum}}, \pi_j^{\mathit{lim}}) \stackrel{?}{=} 1$ *proving sum of selections is in* $[0, L_j]$
  3. *Recomputes contest hashes* $\chi'_j = H(H_E \| j \| \{ct_{j,\ell}\}_{\ell=1}^{n_j})$ *for all* j
  4. *Verifies confirmation code* $H(H_E \| \chi'_1 \| \cdots \| \chi'_{|\mathcal{M}|}) \stackrel{?}{=} H_B$

  Outputs 1 *if all verifications pass, else* 0.
- $\{T_{j,\ell}\}_{j,\ell} \leftarrow \mathsf{Aggregate}(\{B^{(i)}\}_{i=1}^{n_{\mathit{ballots}}})$*: On input set of* n_{ballots} *cast ballots* $\{B^{(i)}\}$ *where* $B^{(i)} = \{\{ct_{j,\ell}^{(i)}\}_\ell\}_j$, *the aggregation algorithm computes for each contest* j *and option* ℓ*: homomorphic tally ciphertext* $T_{j,\ell} = \mathsf{HSTE.HomAdd}(ct_{j,\ell}^{(1)}, \ldots, ct_{j,\ell}^{(n_{\mathit{ballots}})})$ *where all ballots must share identical committee structure (same* $D, R, \mathsf{pk}_{\mathit{eph}}$*), and outputs encrypted tallies* $\{T_{j,\ell}\}_{j,\ell}$.
- $\{\sigma_{i,j,\ell}\}_{j,\ell} \leftarrow \mathsf{PartTally}(\mathsf{sk}_i, \mathsf{id}_i, \{T_{j,\ell}\}_{j,\ell})$*: Takes input guardian* G_i*'s secret key* sk_i, *identity* id_i, *and encrypted tallies* $\{T_{j,\ell}\}_{j,\ell}$, *the partial tallying algorithm computes for each tally ciphertext* $T_{j,\ell}$*: partial decryption* $\sigma_{i,j,\ell} = (i, d_{i,j,\ell}, r_{i,j,\ell}) \leftarrow \mathsf{HSTE.PartDec}(\mathsf{sk}_i, \mathsf{id}_i, T_{j,\ell})$, *and outputs partial tally shares* $\{\sigma_{i,j,\ell}\}_{j,\ell}$.
- $\{t_{j,\ell}\}_{j,\ell} \leftarrow \mathsf{Tally}(\{T_{j,\ell}\}_{j,\ell}, \{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell})$*: On input encrypted tallies* $\{T_{j,\ell}\}_{j,\ell}$ *and partial tally shares* $\{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell}$ *from guardian set* $\mathcal{S}$ *with* $|\mathcal{S}| \geq m/2$, *the tallying algorithm computes for each contest* j *and option* ℓ*: decrypted tally* $t_{j,\ell} \leftarrow \mathsf{HSTE.Dec}(T_{j,\ell}, \{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}})$, *verifies* $t_{j,\ell} \in \mathbb{Z}_{\geq 0}$ *and* $t_{j,\ell} \leq n_{\mathit{ballots}} \cdot R_\ell$ *for sanity, and outputs plaintext tallies* $\{t_{j,\ell}\}_{j,\ell}$.
- $\{0,1\} \leftarrow \mathsf{Verify}(\mathsf{PK}, \{T_{j,\ell}\}_{j,\ell}, \{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell}, \{t_{j,\ell}\}_{j,\ell})$*: Takes input election public key* $\mathsf{PK} = \{(\mathsf{pk}_i, i)\}_{i=1}^N$, *encrypted tallies* $\{T_{j,\ell}\}_{j,\ell}$, *partial shares* $\{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell}$, *and claimed tallies* $\{t_{j,\ell}\}_{j,\ell}$, *the verification algorithm performs for each contest* j *and option* ℓ*:*
  1. *For each guardian* $i \in \mathcal{S}$, *verifies partial decryption correctness using the function* $\mathsf{HSTE.PartVerify}(\mathsf{pk}_i, T_{j,\ell}, \sigma_{i,j,\ell}) \stackrel{?}{=} 1$
  2. *Verifies threshold satisfaction* $|\{i \in \mathcal{S} : \mathsf{HSTE.PartVerify}(\mathsf{pk}_i, T_{j,\ell}, \sigma_{i,j,\ell}) = 1\}| \stackrel{?}{\geq} m/2$
  3. *Recomputes tally* $t'_{j,\ell} \leftarrow \mathsf{HSTE.Dec}(T_{j,\ell}, \{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}})$, *verifies* $t'_{j,\ell} \stackrel{?}{=} t_{j,\ell}$

  Outputs 1 *if all verifications pass for all tallies, else* 0.

The scheme achieves post-quantum security under M-LWE and M-SIS assumptions (via HSTE and lattice-based range proofs), provides silent setup with independent guardian key generation eliminating interactive key ceremony, achieves end-to-end verifiability through lattice-based ballot correctness proofs and HSTE's built-in partial decryption verification, and supports homomorphic tally aggregation with threshold $m/2$ decryption requiring only majority of randomly-selected committee.

A Appendix

A.1 Proof of Theorem 4

Proof. We prove correctness by analyzing each component of the decryption pipeline. Let $(pk_i,sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda,\mathsf{id}_i)$ for all $i \in [N]$, and let $ct = (ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathtt{eph}}) \leftarrow \mathsf{HSTE.Enc}(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu)$ for some $\mu \in R_p$. Denote by $\mathcal{C} \subseteq [N]$ the committee sampled during encryption with $|\mathcal{C}| = m$, and let $\mathcal{S}' = \mathcal{S} \cap \mathcal{C}$ where $|\mathcal{S}'| \geq \tau = \lceil m/2 \rceil$. During encryption, the algorithm constructs an aggregated BFV public key:

$$pk_{\mathtt{bfv},\mathcal{C}} = (\mathbf{a}_0,\mathbf{b}_\mathcal{C}) \quad \text{where} \quad \mathbf{b}_\mathcal{C} := \frac{1}{m}\sum_{i\in\mathcal{C}} \mathbf{b}_i,$$

and $\mathbf{a}_0 = \mathbf{a}_i$ for all $i \in \mathcal{C}$ (common $\mathbf{a}$ component). Each party i's secret key satisfies $\mathbf{b}_i = -(\mathbf{a}_0 \cdot s_i + \mathbf{e}_i)$ for $s_i \leftarrow \chi_{\mathtt{key}}$ and $\mathbf{e}_i \leftarrow \chi_{\mathtt{err}}^k$. Therefore:

$$\mathbf{b}_\mathcal{C} = \frac{1}{m}\sum_{i\in\mathcal{C}} \mathbf{b}_i = \frac{1}{m}\sum_{i\in\mathcal{C}} (-\mathbf{a}_0 \cdot s_i - \mathbf{e}_i) = -\mathbf{a}_0 \cdot \left(\frac{1}{m}\sum_{i\in\mathcal{C}} s_i\right) - \frac{1}{m}\sum_{i\in\mathcal{C}} \mathbf{e}_i =: -\mathbf{a}_0 \cdot s_\mathcal{C} - \mathbf{e}_\mathcal{C},$$

where $s_\mathcal{C} := \frac{1}{m}\sum_{i\in\mathcal{C}} s_i$ is the aggregated secret key and $\mathbf{e}_\mathcal{C} := \frac{1}{m}\sum_{i\in\mathcal{C}} \mathbf{e}_i$ is the aggregated error. The ciphertext $(ct_0,ct_1) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}},\mu)$ satisfies:

$$ct_0 = \mathbf{b}_\mathcal{C} \cdot \mathbf{u} + e_1 + \Delta \cdot \mu \in R_q,$$
$$ct_1 = \mathbf{a}_0 \cdot \mathbf{u} + e_2 \in R_q,$$

for $\mathbf{u} \leftarrow \chi_{\mathtt{key}}$, $e_1,e_2 \leftarrow \chi_{\mathtt{err}}$, and $\Delta = \lfloor q/p \rfloor$. For each party $i \in \mathcal{S}' \subseteq \mathcal{C}$, the partial decryption algorithm computes $K_i = \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{eph}},0,\mathsf{sk}_{\mathtt{nike},i},\mathsf{id}_i)$. By the correctness of GAHDH (Assumption 2), with probability $\geq 1 - \mathsf{negl}(\lambda)$, this equals $K_i = \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{nike},i},\mathsf{id}_i,\mathsf{sk}_{\mathtt{eph}},0)$, which is the same key computed during encryption. Therefore, the dictionary lookup $v_i = (ct_{\mathtt{kyber},i},c_i,h_i) \leftarrow \mathsf{DS.Get}(\mathcal{D},R,K_i)$ succeeds with probability $\geq 1 - 2^{-128}$ (Assumption 4), retrieving the correct tuple stored during encryption. Party i decapsulates $K_{\mathtt{kem},i} \leftarrow \mathsf{Kyber.Decaps}(\mathsf{dk}_{\mathtt{kyber},i},ct_{\mathtt{kyber},i})$, which by δ-correctness of Kyber (Assumption 3) equals the key used during encryption with probability $\geq 1 - \mathsf{negl}(\lambda)$. Hence $r_i = \mathsf{AES}^{-1}_{K_{\mathtt{kem},i}}(c_i)$ recovers the randomness r_i used during encryption. The tag verification $H(i \| r_i) \stackrel{?}{=} h_i$ succeeds with overwhelming probability by the determinism of H (Assumption 5). Each party $i \in \mathcal{S}'$ computes the partial decryption $d_i = ct_1 \cdot s_i = (\mathbf{a}_0 \cdot \mathbf{u} + e_2) \cdot s_i = \mathbf{a}_0 \cdot \mathbf{u} \cdot s_i + e_2 \cdot s_i \in R_q$. The decryption

$\mathsf{PQEG.Setup}(1^\lambda, \mathcal{M}, N)$

for $i \in [1,N]$ **do**
  $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda, i)$
  $(pk_{\mathtt{bfv},i}, \mathsf{pk}_{\mathtt{nike},i}, \mathsf{ek}_{\mathtt{kyber},i}, \mathsf{rlk}_i) \leftarrow \mathsf{pk}_i$
  $(s_i, \mathsf{sk}_{\mathtt{nike},i}, \mathsf{dk}_{\mathtt{kyber},i}) \leftarrow \mathsf{sk}_i$
$\mathsf{PK} \leftarrow \{(\mathsf{pk}_i, i)\}_{i=1}^N, \quad L \leftarrow \max_{\mathtt{contests}} L_j, \quad \ell \leftarrow \lceil \log_2(L) \rceil$
$\mathsf{pp}_{\mathtt{rp}} \leftarrow \mathsf{RangeProof.Setup}(1^\lambda, \ell)$
$H_M \leftarrow H(\mathcal{M}), \quad H_E \leftarrow H(H_M \| \mathsf{PK} \| \mathsf{pp}_{\mathtt{rp}})$
return $(\{(\mathsf{pk}_i, i)\}_{i=1}^N, \{\mathsf{sk}_i\}_{i=1}^N, \mathsf{pp}_{\mathtt{rp}}, H_E)$

$\mathsf{PQEG.Vote}(\mathsf{PK}, \mathcal{M}, \mathsf{pp}_{\mathtt{rp}}, \mathbf{v})$

$\xi_B \leftarrow\!\$ \{0,1\}^\lambda, \quad B \leftarrow \emptyset, \quad \text{proofs} \leftarrow \emptyset, \quad \text{contest\_hashes} \leftarrow \emptyset$
for $j \in [1, |\mathcal{M}|]$ **do**
  $(n_j, L_j) \leftarrow \mathcal{M}[j], \quad \text{encryptions}_j \leftarrow \emptyset$
  $\text{opt\_proofs}_j \leftarrow \emptyset, \quad \text{commitments}_j \leftarrow \emptyset$
  for $\ell \in [1, n_j]$ **do**
    $\xi_{j,\ell} \leftarrow H(H_E \| \xi_B \| j \| \ell)$
    $ct_{j,\ell} \leftarrow \mathsf{HSTE.Enc}(\mathsf{PK}, v_{j,\ell}; \xi_{j,\ell})$
    $(ct_{0,j,\ell}, ct_{1,j,\ell}, \mathcal{D}_B, R_B, \mathsf{pk}_{\mathtt{eph},B}) \leftarrow ct_{j,\ell}$
    $(C_{j,\ell}, \mathbf{r}_{j,\ell}) \leftarrow \mathsf{RangeProof.Commit}(\mathsf{pp}_{\mathtt{rp}}, v_{j,\ell})$
    $\pi_{j,\ell}^{\mathtt{opt}} \leftarrow \mathsf{RangeProof.Prove}(\mathsf{pp}_{\mathtt{rp}}, v_{j,\ell}, C_{j,\ell}, \mathbf{r}_{j,\ell})$
    $\text{encryptions}_j \leftarrow \text{encryptions}_j \cup \{ct_{j,\ell}\}$
    $\text{opt\_proofs}_j \leftarrow \text{opt\_proofs}_j \cup \{\pi_{j,\ell}^{\mathtt{opt}}\}$
    $\text{commitments}_j \leftarrow \text{commitments}_j \cup \{(C_{j,\ell}, \mathbf{r}_{j,\ell})\}$
  $ct_j^{\mathtt{sum}} \leftarrow \mathsf{HSTE.HomAdd}(ct_{j,1}, \ldots, ct_{j,n_j})$
  $\mathbf{r}_j^{\mathtt{sum}} \leftarrow \sum_{\ell=1}^{n_j} \mathbf{r}_{j,\ell}$
  $(C_j^{\mathtt{sum}}, \mathbf{r}_j^{\mathtt{sum}}) \leftarrow \mathsf{RangeProof.Commit}(\mathsf{pp}_{\mathtt{rp}}, \sum_\ell v_{j,\ell})$
  $\pi_j^{\mathtt{lim}} \leftarrow \mathsf{RangeProof.Prove}(\mathsf{pp}_{\mathtt{rp}}, \sum_\ell v_{j,\ell}, C_j^{\mathtt{sum}}, \mathbf{r}_j^{\mathtt{sum}})$
  $\chi_j \leftarrow H(H_E \| j \| \{ct_{j,\ell}\}_{\ell=1}^{n_j})$
  $B \leftarrow B \cup \{\{ct_{j,\ell}\}_{\ell=1}^{n_j}\}$
  $\text{proofs} \leftarrow \text{proofs} \cup \{(\text{opt\_proofs}_j, \pi_j^{\mathtt{lim}})\}$
  $\text{contest\_hashes} \leftarrow \text{contest\_hashes} \cup \{\chi_j\}$
$H_B \leftarrow H(H_E \| \chi_1 \| \cdots \| \chi_{|\mathcal{M}|})$
return (B, H_B, proofs)

$\mathsf{PQEG.Aggregate}(\{B^{(i)}\}_{i=1}^{n_{\mathtt{ballots}}})$

$\text{tallies} \leftarrow \emptyset$
for $j \in [1, |\mathcal{M}|]$ **do**
  for $\ell \in [1, n_j]$ **do**
    $T_{j,\ell} \leftarrow \mathsf{HSTE.HomAdd}(ct_{j,\ell}^{(1)}, \ldots, ct_{j,\ell}^{(n_{\mathtt{ballots}})})$
    $\text{tallies} \leftarrow \text{tallies} \cup \{((j,\ell), T_{j,\ell})\}$
return $\{T_{j,\ell}\}_{j,\ell}$

$\mathsf{PQEG.Valid}(\mathsf{PK}, \mathcal{M}, \mathsf{pp}_{\mathtt{rp}}, B, \{\pi_j\}_j, H_B)$

$\mathsf{require}(\text{BallotStructure}(B) = \mathcal{M})$
$\text{contest\_hashes}' \leftarrow \emptyset$
for $j \in [1, |\mathcal{M}|]$ **do**
  $(n_j, L_j) \leftarrow \mathcal{M}[j]$
  $(\{\pi_{j,\ell}^{\mathtt{opt}}\}_{\ell=1}^{n_j}, \pi_j^{\mathtt{lim}}) \leftarrow \pi_j$
  for $\ell \in [1, n_j]$ **do**
    $C_{j,\ell} \leftarrow \mathsf{ExtractCommitment}(\pi_{j,\ell}^{\mathtt{opt}})$
    $\mathsf{require}(\mathsf{RangeProof.Verify}(\mathsf{pp}_{\mathtt{rp}}, -, C_{j,\ell}, \pi_{j,\ell}^{\mathtt{opt}}) = 1)$
  $C_j^{\mathtt{sum}} \leftarrow \mathsf{ExtractCommitment}(\pi_j^{\mathtt{lim}})$
  $\mathsf{require}(\mathsf{RangeProof.Verify}(\mathsf{pp}_{\mathtt{rp}}, -, C_j^{\mathtt{sum}}, \pi_j^{\mathtt{lim}}) = 1)$
  $\{ct_{j,\ell}\}_{\ell=1}^{n_j} \leftarrow B[j]$
  $\chi'_j \leftarrow H(H_E \| j \| \{ct_{j,\ell}\}_{\ell=1}^{n_j})$
  $\text{contest\_hashes}' \leftarrow \text{contest\_hashes}' \cup \{\chi'_j\}$
$H'_B \leftarrow H(H_E \| \chi'_1 \| \cdots \| \chi'_{|\mathcal{M}|})$
$\mathsf{require}(H'_B = H_B)$
return 1

$\mathsf{PQEG.PartTally}(\mathsf{sk}_i, \mathsf{id}_i, \{T_{j,\ell}\}_{j,\ell})$

$\text{partial\_shares} \leftarrow \emptyset$
for $(j,\ell) \in \text{AllContestOptions}(\mathcal{M})$ **do**
  $\sigma_{i,j,\ell} \leftarrow \mathsf{HSTE.PartDec}(\mathsf{sk}_i, \mathsf{id}_i, T_{j,\ell})$
  $(i, d_{i,j,\ell}, r_{i,j,\ell}) \leftarrow \sigma_{i,j,\ell}$
  $\text{partial\_shares} \leftarrow \text{partial\_shares} \cup \{((j,\ell), \sigma_{i,j,\ell})\}$
return $\{\sigma_{i,j,\ell}\}_{j,\ell}$

$\mathsf{PQEG.Tally}(\{T_{j,\ell}\}_{j,\ell}, \{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell})$

$\mathsf{require}(|\mathcal{S}| \geq m/2)$
$\text{plaintext\_tallies} \leftarrow \emptyset$
for $(j,\ell) \in \text{AllContestOptions}(\mathcal{M})$ **do**
  $t_{j,\ell} \leftarrow \mathsf{HSTE.Dec}(T_{j,\ell}, \{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}})$
  $\mathsf{require}(t_{j,\ell} \in \mathbb{Z}_{\geq 0} \wedge t_{j,\ell} \leq n_{\mathtt{ballots}} \cdot R_\ell)$
  $\text{plaintext\_tallies} \leftarrow \text{plaintext\_tallies} \cup \{((j,\ell), t_{j,\ell})\}$
return $\{t_{j,\ell}\}_{j,\ell}$

$\mathsf{PQEG.Verify}(\mathsf{PK}, \{T_{j,\ell}\}_{j,\ell}, \{\{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}}\}_{j,\ell}, \{t_{j,\ell}\}_{j,\ell})$

for $(j,\ell) \in \text{AllContestOptions}(\mathcal{M})$ **do**
$\mathcal{S}_v \leftarrow \emptyset$
for $i \in \mathcal{S}$ **do**
  if $\mathsf{HSTE.PartVerify}(\mathsf{pk}_i, T_{j,\ell}, \sigma_{i,j,\ell}) = 1$ **then**
    $\mathcal{S}_v \leftarrow \mathcal{S}_v \cup \{i\}$
$\mathsf{require}(|\mathcal{S}_v| \geq m/2)$
$t'_{j,\ell} \leftarrow \mathsf{HSTE.Dec}(T_{j,\ell}, \{\sigma_{i,j,\ell}\}_{i \in \mathcal{S}})$
$\mathsf{require}(t'_{j,\ell} = t_{j,\ell})$
return 1

Fig. 3: Post-Quantum ElectionGuard PQEG with Silent Setup

algorithm computes Lagrange coefficients $\{\lambda_i\}_{i\in\mathcal{S}_v}$ over $\mathbb{Z}_q$ for $\mathcal{S}_v\subseteq\mathcal{S}'$ (the verified subset), where $|\mathcal{S}_v|\geq\tau$. These coefficients satisfy:

$$\sum_{i\in\mathcal{S}_v}\lambda_i\cdot i^j=\begin{cases}1 & \text{if } j=0,\\ 0 & \text{if } 1\leq j<|\mathcal{S}_v|.\end{cases}$$

The aggregated partial decryption is:

$$\begin{aligned}d_{\mathsf{agg}}&=\frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot d_i\\&=\frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot(\mathbf{a}_0\cdot\mathbf{u}\cdot s_i+e_2\cdot s_i)\\&=\mathbf{a}_0\cdot\mathbf{u}\cdot\left(\frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot s_i\right)+e_2\cdot\left(\frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot s_i\right).\end{aligned}$$

Since $|\mathcal{S}_v|\geq\tau=\lceil m/2\rceil>m/2$, the Lagrange interpolation property ensures:

$$\frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot s_i=\frac{1}{m}\sum_{i\in\mathcal{C}}s_i=s_{\mathcal{C}},$$

where the second equality holds because the interpolation polynomial evaluated at degree-0 recovers the constant term (average of all committee secrets). Therefore $d_{\mathsf{agg}}=\mathbf{a}_0\cdot\mathbf{u}\cdot s_{\mathcal{C}}+e_2\cdot s_{\mathcal{C}}=ct_1\cdot s_{\mathcal{C}}$. The final decryption computes:

$$\begin{aligned}\tilde{\mu}&=ct_0-d_{\mathsf{agg}}\\&=(\mathbf{b}_{\mathcal{C}}\cdot\mathbf{u}+e_1+\Delta\cdot\mu)-ct_1\cdot s_{\mathcal{C}}\\&=(-\mathbf{a}_0\cdot s_{\mathcal{C}}-\mathbf{e}_{\mathcal{C}})\cdot\mathbf{u}+e_1+\Delta\cdot\mu-(\mathbf{a}_0\cdot\mathbf{u}+e_2)\cdot s_{\mathcal{C}}\\&=-\mathbf{a}_0\cdot\mathbf{u}\cdot s_{\mathcal{C}}-\mathbf{e}_{\mathcal{C}}\cdot\mathbf{u}+e_1+\Delta\cdot\mu-\mathbf{a}_0\cdot\mathbf{u}\cdot s_{\mathcal{C}}-e_2\cdot s_{\mathcal{C}}\\&=\Delta\cdot\mu+\mathsf{Err},\end{aligned}$$

where $\mathsf{Err}:=-\mathbf{e}_{\mathcal{C}}\cdot\mathbf{u}+e_1-e_2\cdot s_{\mathcal{C}}$ is the total noise term.

By Assumption 1, with overwhelming probability $\|\mathsf{Err}\|_\infty<\frac{q}{2p}=\frac{\Delta}{2}$, because $\|\mathbf{e}_{\mathcal{C}}\|_\infty\leq\frac{1}{m}\sum_{i\in\mathcal{C}}\|\mathbf{e}_i\|_\infty=O(\sigma_{\mathtt{err}})$ (where $\sigma_{\mathtt{err}}$ is the error distribution parameter), and similarly for other noise terms. Therefore:

$$\mu'=\left\lfloor\frac{p}{q}\cdot\tilde{\mu}\right\rceil\bmod p=\left\lfloor\frac{p}{q}\cdot(\Delta\cdot\mu+\mathsf{Err})\right\rceil\bmod p=\mu,$$

since $\frac{p}{q}\cdot\Delta\approx1$ and $|\frac{p}{q}\cdot\mathsf{Err}|<1/2$. The decryption fails only if one of the following occurs:

1. NIKE key agreement fails: probability $\leq m\cdot\mathsf{negl}(\lambda)$,
2. Dictionary lookup fails: probability $\leq m\cdot2^{-128}$,
3. Kyber decapsulation fails: probability $\leq m\cdot\mathsf{negl}(\lambda)$,
4. BFV noise exceeds bound: probability $\leq\mathsf{negl}(\lambda)$,

5. Hash collision occurs: probability $\leq \mathsf{negl}(\lambda)$.

By union bound, the total failure probability is:

$$\Pr[\mu' \neq \mu] \leq 3m \cdot \mathsf{negl}(\lambda) + m \cdot 2^{-128} + \mathsf{negl}(\lambda) = \mathsf{negl}(\lambda).$$

Thus, $\Pr[\mu' = \mu] \geq 1 - \mathsf{negl}(\lambda)$, completing the proof. □

A.2 Proof of Theorem 5

Proof. We establish compactness by bounding the size of each component in the ciphertext and public key structures. Starting by **Ciphertext Compactness**, a ciphertext has the form $ct = (ct_0, ct_1, \mathcal{D}, R, \mathsf{pk}_{\mathsf{eph}})$ where:

(i) BFV ciphertext components $(ct_0, ct_1) \in R_q^2$: $|ct_0| + |ct_1| = 2 \cdot n \cdot \lceil \log_2 q \rceil$ bits. For security parameter λ, standard BFV parameters require $n = O(\lambda)$ (typically $n = 2^k$ for $k = 11,12,13$) and $\log q = O(\lambda)$ (typically $\log q \approx 109$ for 128-bit security). Thus $|ct_0| + |ct_1| = O(\lambda^2)$ bits.

(ii) PTRHash dictionary $\mathcal{D}$: By Definition 11, for m entries with key space $\{0,1\}^k$ and value space $\{0,1\}^v$, the dictionary size satisfies $|\mathcal{D}| \leq m \cdot v + \frac{m}{3}$ bits, where $v = |ct_{\mathtt{kyber}}| + |c| + |h|$ for each committee member. We bound each component:
   - $|ct_{\mathtt{kyber}}|$: For $\mathsf{Kyber512}$, $|ct_{\mathtt{kyber}}| = 768$ bytes $= 6144$ bits.
   - $|c| = |r| = \lambda$ bits (encrypted randomness).
   - $|h| = 256$ bits (hash tag output).

   Therefore $v = 6144 + \lambda + 256 = O(\lambda)$ bits, *and* $|\mathcal{D}| = m \cdot O(\lambda) + \frac{m}{3} = O(m \cdot \lambda)$ bits. Since $m = \mathrm{poly}(\lambda)$ by assumption, we have $|\mathcal{D}| = O(\mathrm{poly}(\lambda) \cdot \lambda) = O(\mathrm{poly}(\lambda))$ bits.

(iii) Dictionary seed $R \in \{0,1\}^{128}$: $|R| = 128$ bits $= O(1)$.

(iv) Ephemeral NIKE public key $\mathsf{pk}_{\mathsf{eph}}$: For GAHDH based on $\mathsf{CSIDH\text{-}512}$ (Definition 10), the public key is a compressed j-invariant $|\mathsf{pk}_{\mathsf{eph}}| = 64$ bytes $= 512$ bits $= O(\lambda)$.

Combining all components:

$$\begin{aligned} |ct| &= |ct_0| + |ct_1| + |\mathcal{D}| + |R| + |\mathsf{pk}_{\mathsf{eph}}| \\ &= O(\lambda^2) + O(\mathrm{poly}(\lambda)) + O(1) + O(\lambda) \\ &= O(\mathrm{poly}(\lambda)) \text{ bits.} \end{aligned}$$

Define $\mathsf{poly}_1(\lambda) := C_1 \cdot (\lambda^2 + m(\lambda) \cdot \lambda + \lambda + 128)$ for an appropriate constant $C_1 > 0$ and $m(\lambda) = \mathrm{poly}(\lambda)$. Then $|ct| \leq \mathsf{poly}_1(\lambda)$.
The ciphertext size depends on m (the committee size), *not on* N (the total number of parties). This is the key compactness property: an encryption to N parties produces a ciphertext of size polynomial in λ and independent of N, achieved through the committee sampling mechanism.

The next part to check is **Public Key Compactness**. Each party $i \in [N]$ has a public key $pk_i = (pk_{\mathtt{bfv},i}, \mathsf{pk}_{\mathtt{nike},i}, \mathsf{ek}_{\mathtt{kyber},i}, \mathsf{rlk}_i)$ where:

(i) BFV public key $pk_{\mathtt{bfv},i}=(\mathbf{a},\mathbf{b}_i)\in R_q^{k+1}$: $|pk_{\mathtt{bfv},i}|=(k+1)\cdot n\cdot\lceil\log_2 q\rceil$ bits. For standard BFV parameters, $k=O(1)$ (typically $k=1$ or 2), so: $|pk_{\mathtt{bfv},i}|=O(n\cdot\log q)=O(\lambda^2)$ bits.
(ii) NIKE public key $\mathsf{pk}_{\mathtt{nike},i}$: $|\mathsf{pk}_{\mathtt{nike},i}|=512$ bits$=O(\lambda)$.
(iii) Kyber encryption key $\mathsf{ek}_{\mathtt{kyber},i}$: $|\mathsf{ek}_{\mathtt{kyber},i}|=800$ bytes$=6400$ bits$=O(\lambda)$.
(iv) Relinearization key rlk_i: For BFV, the relinearization key consists of encryptions of powers of the secret key. Using gadget decomposition with base B and $\ell=\lceil\log_B q\rceil$ digits:

$$|\mathsf{rlk}_i|=O(\ell\cdot n\cdot\log q)=O\left(\frac{\log q}{\log B}\cdot n\cdot\log q\right)=O(n\cdot\log^2 q)=O(\lambda^3)\text{ bits.}$$

Combining all components:

$$\begin{aligned}|pk_i|&=|pk_{\mathtt{bfv},i}|+|\mathsf{pk}_{\mathtt{nike},i}|+|\mathsf{ek}_{\mathtt{kyber},i}|+|\mathsf{rlk}_i|\\&=O(\lambda^2)+O(\lambda)+O(\lambda)+O(\lambda^3)\\&=O(\lambda^3)\text{ bits.}\end{aligned}$$

Define $\mathsf{poly}_2(\lambda):=C_2\cdot\lambda^3$ for an appropriate constant $C_2>0$. Then for all $i\in[N]$: $|pk_i|\leq\mathsf{poly}_2(\lambda)$. And finally, proceeding with **Concrete Instantiation**, for 128-bit security with standard parameters:

- $n=2048$, $\log q=109$, $p=2^{16}$ (BFV)
- $m=500$ (committee size)
- CSIDH-512 (64-byte public keys)
- Kyber512 (800-byte public keys, 768-byte ciphertexts)

The concrete ciphertext size is:

$$\begin{aligned}|ct|&\approx 2\cdot 2048\cdot 109+500\cdot(6144+128+256)+128+512\\&\approx 446{,}464+3{,}264{,}000+640\\&\approx 3.71\text{ MB}=\mathsf{poly}_1(128).\end{aligned}$$

The concrete public key size is:

$$\begin{aligned}|pk_i|&\approx 2\cdot 2048\cdot 109+512+6400+2048\cdot 109^2\\&\approx 446{,}464+6912+24{,}354{,}112\\&\approx 24.8\text{ MB}=\mathsf{poly}_2(128).\end{aligned}$$

Both quantities are polynomial in $\lambda=128$, confirming compactness. Therefore, HSTE satisfies the compactness property (Definition 3) with explicit polynomial bounds $\mathsf{poly}_1(\lambda)=O(\mathrm{poly}(\lambda))$ and $\mathsf{poly}_2(\lambda)=O(\lambda^3)$. □

A.3 Proof of Theorem 6

Proof. We prove that HSTE satisfies IND-CCA security (Definition 6). Let $\mathcal{A}$ be a PPT adversary attacking the IND-CCA security of HSTE with advantage

$\epsilon = \mathsf{Adv}^{\mathsf{TPKE}}_{\mathcal{A}}(\lambda)$. We construct a sequence of games $\{\mathsf{Game}_j\}_{j=0}^{4}$ where Game_0 corresponds to $\mathsf{Exp}^{\mathsf{TPKE}}_{\mathcal{A},0}(1^\lambda)$ and show that Game_4 is independent of the challenge bit b, implying $\epsilon \leq \mathsf{negl}(\lambda)$.

Game 0: This is the real experiment $\mathsf{Exp}^{\mathsf{TPKE}}_{\mathcal{A},0}(1^\lambda)$ where the challenge ciphertext encrypts m_0^*:

1. $\mathcal{A}$ outputs an access structure $\mathbb{A}$ (implicitly defining threshold $\tau = \lceil m/2 \rceil$)
2. For all $i \in [N]$: $(pk_i, sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda, \mathsf{id}_i)$
3. $\mathcal{A}$ receives $\{pk_i\}_{i\in[N]}$ and outputs corruption set $S^* \subseteq [N]$ with $|S^*| < \tau$
4. $\mathcal{A}$ receives $\{sk_i\}_{i\in S^*}$ and has access to $\mathcal{O}_{\mathsf{PartDec}}(\cdot, i)$ for $i \in [N] \setminus S^*$
5. $\mathcal{A}$ outputs challenge messages (m_0^*, m_1^*)
6. Challenger computes $ct^* = (ct_0^*, ct_1^*, \mathcal{D}^*, R^*, \mathsf{pk}^*_{\mathsf{eph}}) \leftarrow \mathsf{HSTE.Enc}(\{(pk_i, \mathsf{id}_i)\}_{i=1}^{N}, m_0^*)$ with committee $\mathcal{C}^* \subseteq [N]$, $|\mathcal{C}^*| = m$
7. $\mathcal{A}$ continues with $\mathcal{O}_{\mathsf{PartDec}}$ access (except on (ct^*, i) for any i)
8. $\mathcal{A}$ outputs guess $b' \in \{0,1\}$

Let $\Pr[\mathsf{Game}_0] := \Pr[b' = 0]$ be the probability that $\mathcal{A}$ outputs 0 in this game.

Game 1: Modify Game_0 by replacing the NIKE shared keys for honest parties in the challenge committee. Let $\mathcal{H}^* := \mathcal{C}^* \setminus S^*$ denote the honest parties in the challenge committee (note $|\mathcal{H}^*| \geq m - |S^*| > m - \tau \geq m/2$). For each $j \in \mathcal{H}^*$:

- Generate ephemeral NIKE key pair $(\mathsf{pk}^*_{\mathsf{eph}}, \mathsf{sk}^*_{\mathsf{eph}}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda, 0)$ as before
- Instead of computing $K_j = \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{nike},j}, \mathsf{id}_j, \mathsf{sk}^*_{\mathsf{eph}}, 0)$, sample $K_j \leftarrow\!\!\$ \{0,1\}^{512}$ uniformly at random
- Continue constructing dictionary entries (K_j, v_j) where $v_j = (ct_{\mathtt{kyber},j}, c_j, h_j)$ using the random K_j

For corrupted parties $j \in S^* \cap \mathcal{C}^*$, compute K_j honestly as before (the adversary could compute these anyway using $\mathsf{sk}_{\mathtt{nike},j}$).

Lemma 1. *If* GAHDH *satisfies one-pair malicious-key security under* GA-StCDH *(Theorem 3), then* $|\Pr[\mathsf{Game}_1] - \Pr[\mathsf{Game}_0]| \leq m \cdot \mathsf{Adv}^{\mathsf{GA\text{-}StCDH}}_{\mathcal{B}_1}(\lambda) \leq \mathsf{negl}(\lambda)$.

Proof of Lemma 1. We construct a reduction $\mathcal{B}_1$ to the one-pair malicious-key security of GAHDH. $\mathcal{B}_1$ receives a challenge public key $\mathsf{pk}_{\mathtt{chal}}$ and must distinguish between $K_{\text{real}} = \mathsf{GAHDH.SharedKey}(\mathsf{pk}_{\mathtt{chal}}, \mathsf{id}_{\mathtt{chal}}, \mathsf{sk}_{\mathtt{adv}}, \mathsf{id}_{\mathtt{adv}})$ and $K_{\text{rand}} \leftarrow\!\!\$ \{0,1\}^{512}$, where $\mathsf{sk}_{\mathtt{adv}}$ is chosen by $\mathcal{B}_1$. The Adversary $\mathcal{B}_1$ simulates the HSTE game for $\mathcal{A}$ as follows:

1. Receive $\mathcal{A}$'s access structure $\mathbb{A}$
2. For all $i \in [N]$: generate $(pk_i, sk_i) \leftarrow \mathsf{HSTE.Gen}(1^\lambda, \mathsf{id}_i)$ honestly
3. Receive $\mathcal{A}$'s corruption set S^* and provide $\{sk_i\}_{i\in S^*}$
4. Answer $\mathcal{O}_{\mathsf{PartDec}}$ queries honestly using secret keys
5. Receive challenge messages (m_0^*, m_1^*)
6. Generate challenge ciphertext:
   - Sample committee $\mathcal{C}^* \leftarrow\!\!\$ \binom{[N]}{m}$
   - Choose random index $j^* \leftarrow\!\!\$ \mathcal{C}^* \setminus S^*$ (this is the "test" party)
   - Generate $(\mathsf{sk}^*_{\mathsf{eph}}, \mathsf{pk}^*_{\mathsf{eph}}) \leftarrow \mathsf{GAHDH.Gen}(1^\lambda, 0)$ and set $\mathsf{sk}_{\mathtt{adv}} = \mathsf{sk}^*_{\mathsf{eph}}$

- For $j = j^*$: use $\mathsf{pk}_{\mathtt{chal}}$ as $\mathsf{pk}_{\mathtt{nike},j^*}$ and the challenge key K (either K_{real} or K_{rand}) as K_{j^*}
- For other $j \in \mathcal{H}^* \setminus \{j^*\}$: sample $K_j \leftarrow\!\!\$ \{0,1\}^{512}$ randomly
- For $j \in S^* \cap \mathcal{C}^*$: compute K_j honestly
- Construct dictionary $\mathcal{D}^*$ using these keys
- Encrypt BFV portion: $(ct_0^*, ct_1^*) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, m_0^*)$

7. Continue answering $\mathcal{O}_{\mathsf{PartDec}}$ queries (with restriction on ct^*)
8. Receive $\mathcal{A}$'s output b' and output b' to the GAHDH challenger

If $K = K_{\mathrm{real}}$, then $\mathcal{B}_1$ perfectly simulates Game_0. If $K = K_{\mathrm{rand}}$, then $\mathcal{B}_1$ simulates a hybrid where K_{j^*} is random while other honest parties' keys follow Game_0 distribution. By a standard hybrid argument over $|\mathcal{H}^*| \leq m$ parties, the total distinguishing advantage is bounded by $m \cdot \mathsf{Adv}^{\mathsf{GA\text{-}StCDH}}_{\mathcal{B}_1}(\lambda)$. □

Game 2: Modify Game_1 by replacing the Kyber ciphertexts and encrypted randomness for honest committee members. For each $j \in \mathcal{H}^*$:

- Sample $(ct_{\mathtt{kyber},j}, K_{\mathtt{kem},j}) \leftarrow \mathsf{Kyber.Encaps}(\mathsf{ek}_{\mathtt{kyber},j})$ as before
- Instead of encrypting real randomness r_j, sample $c_j \leftarrow\!\!\$ \{0,1\}^{\lambda+128}$ uniformly at random (AES ciphertext length)
- Compute $h_j = H(j \| r_j)$ as before (but note r_j is never used in the ciphertext except via h_j)
- Set $v_j = (ct_{\mathtt{kyber},j}, c_j, h_j)$ in the dictionary

Lemma 2. *If* Kyber *satisfies IND-CCA2 security under* MLWE *(Theorem 2) and* AES *is a secure pseudorandom function, then:*

$$|\Pr[\mathsf{Game}_2] - \Pr[\mathsf{Game}_1]| \leq m \cdot \left(\mathsf{Adv}^{\mathsf{IND\text{-}CCA2}}_{\mathsf{Kyber},\mathcal{B}_2}(\lambda) + \mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{AES},\mathcal{B}_2'}(\lambda) \right) \leq \mathsf{negl}(\lambda).$$

Proof of Lemma 2. For each honest party $j \in \mathcal{H}^*$, the value $c_j = \mathsf{AES}_{K_{\mathtt{kem},j}}(r_j)$ encrypts r_j under key $K_{\mathtt{kem},j}$. By IND-CCA2 security of Kyber, the key $K_{\mathtt{kem},j}$ is indistinguishable from random given $ct_{\mathtt{kyber},j}$, even to an adversary with decapsulation oracle access (which $\mathcal{A}$ has via $\mathcal{O}_{\mathsf{PartDec}}$ for parties not in S^*, but not for ct^*). Once $K_{\mathtt{kem},j}$ is indistinguishable from random, by PRF security of AES, the ciphertext $c_j = \mathsf{AES}_{K_{\mathtt{kem},j}}(r_j)$ is indistinguishable from a random string of the same length.

The reduction $\mathcal{B}_2$ embeds a Kyber IND-CCA2 challenge for a random honest party $j^* \in \mathcal{H}^*$, simulates the rest of the game honestly, and uses $\mathcal{A}$'s output to distinguish. A hybrid argument over $|\mathcal{H}^*| \leq m$ parties gives the stated bound. □

Game 3: Modify Game_2 by changing the BFV component of the challenge ciphertext:

- Compute aggregated public key $pk_{\mathtt{bfv},\mathcal{C}^*} = (\mathbf{a}_0, \mathbf{b}_{\mathcal{C}^*})$ as before
- Instead of $(ct_0^*, ct_1^*) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, m_0^*)$, compute $(ct_0^*, ct_1^*) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, 0)$
- Keep all other components of ct^* identical

Lemma 3. *If* BFV *satisfies IND-CPA security under* RLWE *(Theorem 1), then:*

$$|\Pr[\mathsf{Game}_3] - \Pr[\mathsf{Game}_2]| \leq \mathsf{Adv}^{\mathsf{RLWE}}_{\mathcal{B}_3}(\lambda) \leq \mathsf{negl}(\lambda).$$

Proof of Lemma 3. We construct a reduction $\mathcal{B}_3$ to the IND-CPA security of BFV under RLWE. The key challenge is that $\mathcal{A}$ has access to partial decryptions via $\mathcal{O}_{\mathsf{PartDec}}$, which could potentially leak information about the BFV secret keys. However, observe that:

1. For corrupted parties $j \in S^* \cap \mathcal{C}^*$, the adversary already has $sk_j = s_j$, so partial decryptions $d_j = ct_1^* \cdot s_j$ provide no additional information.
2. For honest parties $j \in \mathcal{H}^*$, the partial decryption query $\mathcal{O}_{\mathsf{PartDec}}(ct^*, j)$ is explicitly forbidden by the IND-CCA experiment.
3. For parties $j \notin \mathcal{C}^*$, the dictionary lookup $\mathsf{DS.Get}(\mathcal{D}^*, R^*, K_j)$ returns $\perp$ (since j was not in the committee), so $\mathsf{HSTE.PartDec}(sk_j, \mathsf{id}_j, ct^*)$ returns $\perp$ without using s_j.
4. For other ciphertexts $ct \neq ct^*$, the adversary can query $\mathcal{O}_{\mathsf{PartDec}}(ct, j)$ for any $j \in [N] \setminus S^*$. These queries involve $d_j = ct_1 \cdot s_j$ where ct_1 is part of $ct \neq ct^*$. Since we are only changing ct^*, these oracle responses remain consistent.

Therefore, $\mathcal{B}_3$ can simulate the IND-CCA game for $\mathcal{A}$ by:

- Receiving BFV challenge $(pk_{\mathtt{bfv}}, ct^*_{\mathtt{bfv}})$ which encrypts either m_0^* or 0
- Generating honest parties' BFV secret keys $\{s_j\}_{j \in \mathcal{H}^*}$ honestly[2]
- Computing aggregated key as $pk_{\mathtt{bfv},\mathcal{C}^*} = (\mathbf{a}, \mathbf{b}_{\mathcal{C}^*})$ where $\mathbf{b}_{\mathcal{C}^*} = \frac{1}{m} \sum_{j \in \mathcal{C}^*} \mathbf{b}_j$
- Using $ct^*_{\mathtt{bfv}}$ as the BFV component of ct^*
- Answering $\mathcal{O}_{\mathsf{PartDec}}(ct, j)$ for $ct \neq ct^*$ honestly using knowledge of s_j

The critical observation is that even though adversary knows $\{s_j\}_{j \in S^* \cap \mathcal{C}^*}$ (at most $|S^*| < \tau < m/2$ keys), the aggregated secret $s_{\mathcal{C}^*} = \frac{1}{m} \sum_{j \in \mathcal{C}^*} s_j$ still has at least $|\mathcal{H}^*| > m/2$ components unknown to the adversary. By the security of BFV (which reduces to RLWE), encryption under the aggregated key remains indistinguishable even with partial knowledge. In fact, given $(\mathbf{a}, \mathbf{a} \cdot s + e)$ and partial leakage of s, the distribution remains pseudorandom as long as sufficient entropy remains in s. With $|\mathcal{H}^*| \geq m/2$ unknown components, the min-entropy of $s_{\mathcal{C}^*}$ is $\Omega(m \cdot \log q) = \Omega(\mathrm{poly}(\lambda))$, sufficient for RLWE security. Thus, $\mathcal{B}_3$'s advantage in distinguishing $\mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, m_0^*)$ from $\mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, 0)$ is bounded by $\mathsf{Adv}^{\mathsf{RLWE}}_{\mathcal{B}_3}(\lambda)$. □

Game 4: In Game_3, the challenge ciphertext ct^* is:

- $(ct_0^*, ct_1^*) \leftarrow \mathsf{BFV.Enc}(pk_{\mathtt{bfv},\mathcal{C}^*}, 0)$ - independent of both m_0^* and m_1^*
- For $j \in \mathcal{H}^*$: K_j is uniformly random (Game_1), c_j is uniformly random (Game_2)
- For $j \in S^* \cap \mathcal{C}^*$: entries are computed using adversary's keys, but reveal nothing about m_0^* or m_1^* since BFV component encrypts 0
- Tags $h_j = H(j \| r_j)$ are random oracle outputs, independent of messages
- Dictionary $\mathcal{D}^*$ is constructed from these components
- Ephemeral key $\mathsf{pk}^*_{\mathsf{eph}}$ is independently generated

[2] IND-CPA security of BFV holds even when adversary knows some of the aggregated secret key components, as long as at least one component is hidden

Therefore, the entire challenge ciphertext ct^* is distributed identically regardless of whether m_0^* or m_1^* was chosen. Thus $\Pr[\mathsf{Game}_3]=\frac{1}{2}$. By the triangle inequality:

$$\begin{aligned}
\mathsf{Adv}_{\mathcal{A}}^{\mathsf{TPKE}}(\lambda) &= \left|\Pr[\mathsf{Exp}_{\mathcal{A},0}^{\mathsf{TPKE}}=1]-\Pr[\mathsf{Exp}_{\mathcal{A},1}^{\mathsf{TPKE}}=1]\right| \\
&= \left|\Pr[\mathsf{Game}_0]-\frac{1}{2}\right| \quad \text{(by symmetry of } b=0 \text{ and } b=1) \\
&\le |\Pr[\mathsf{Game}_0]-\Pr[\mathsf{Game}_1]|+|\Pr[\mathsf{Game}_1]-\Pr[\mathsf{Game}_2]| \\
&\quad +|\Pr[\mathsf{Game}_2]-\Pr[\mathsf{Game}_3]|+\left|\Pr[\mathsf{Game}_3]-\frac{1}{2}\right| \\
&\le m\cdot\mathsf{Adv}_{\mathcal{B}_1}^{\mathsf{GA\text{-}StCDH}}(\lambda)+m\cdot\left(\mathsf{Adv}_{\mathsf{Kyber},\mathcal{B}_2}^{\mathsf{IND\text{-}CCA2}}(\lambda)+\mathsf{Adv}_{\mathsf{AES},\mathcal{B}_2'}^{\mathsf{PRF}}(\lambda)\right) \\
&\quad +\mathsf{Adv}_{\mathcal{B}_3}^{\mathsf{RLWE}}(\lambda)+0 \\
&= \mathsf{negl}(\lambda).
\end{aligned}$$

Since $m=\mathrm{poly}(\lambda)$ and each individual advantage is negligible by Theorems 1, 2, and 3, the total advantage is negligible. Therefore, HSTE satisfies IND-CCA security (Definition 6). □

A.4 Proof of Theorem 7

Proof. We prove robustness by showing that any adversary $\mathcal{A}$ that produces a verifying but incorrect partial decryption can be used to break one of the underlying assumptions. Let $(pk_i,sk_i)\leftarrow\mathsf{HSTE.Gen}(1^\lambda,\mathsf{id}_i)$ for all $i\in[N]$, and let $ct=(ct_0,ct_1,\mathcal{D},R,\mathsf{pk}_{\mathsf{eph}})\leftarrow\mathsf{HSTE.Enc}(\{(pk_i,\mathsf{id}_i)\}_{i=1}^N,\mu)$ for some $\mu\in R_p$. Denote by $\mathcal{C}\subseteq[N]$ the committee sampled during encryption with $|\mathcal{C}|=m$. The adversary $\mathcal{A}$ outputs (i^*,σ^*) where $\sigma^*=(i^*,d^*,r^*)$ is a candidate partial decryption. The honest partial decryption is $\sigma_{i^*}=(i^*,d_{i^*},r_{i^*})\leftarrow\mathsf{HSTE.PartDec}(sk_{i^*},\mathsf{id}_{i^*},ct)$. For σ^* to pass verification, we require:

$$\mathsf{HSTE.PartVerify}(pk_{i^*},ct,\sigma^*)=1 \iff H(i^*\|r^*)=h_{i^*},$$

where h_{i^*} is the tag stored in the dictionary at key K_{i^*}. For robustness to fail, we need both $\sigma^*\neq\sigma_{i^*}$ and $H(i^*\|r^*)=h_{i^*}$.

Game 0: This is the actual robustness experiment. Define:

$$\mathsf{Win}_0 := \text{Event that } b_1\wedge b_2=1 \text{ in the robustness experiment.}$$

We want to show $\Pr[\mathsf{Win}_0]\le\mathsf{negl}(\lambda)$.

Game 1: Modify Game 0 by aborting unless $i^*\in\mathcal{C}$. If $i^*\notin\mathcal{C}$, then $\mathsf{DS.Get}(\mathcal{D},R,K_{i^*})=\perp$ during honest partial decryption, so $\sigma_{i^*}=\perp$. In this case, if $\sigma^*\neq\perp$, the adversary must have either:

- Found a collision in the dictionary (breaking Assumption 4), or
- Guessed the correct NIKE shared key K_{i^*} (breaking Assumption 3).

By union bound $|\Pr[\mathsf{Win}_1]-\Pr[\mathsf{Win}_0]| \leq 2^{-128}+\mathsf{Adv}^{\mathsf{GA\text{-}StCDH}}_{\mathcal{B}_1}(\lambda) \leq \mathsf{negl}(\lambda)$, where $\mathcal{B}_1$ is a reduction to GA-StCDH. Thus, we assume $i^* \in \mathcal{C}$ in subsequent games.

Game 2: Given that $i^* \in \mathcal{C}$ and $\sigma^* = (i^*, d^*, r^*)$ passes verification with $H(i^* \| r^*) = h_{i^*}$, we analyze two exhaustive cases:

Case 2a: $r^* \neq r_{i^*}$. Since $H(i^* \| r^*) = h_{i^*} = H(i^* \| r_{i^*})$ but $r^* \neq r_{i^*}$, the adversary has found a collision in the random oracle H for inputs $i^* \| r^*$ and $i^* \| r_{i^*}$. By Assumption 1 (random oracle model), the probability of finding such a collision is: $\Pr[\text{Case 2a}] \leq \frac{Q_H^2}{2^{257}}$, where $Q_H = \text{poly}(\lambda)$ is the number of random oracle queries made by $\mathcal{A}$. For any polynomial Q_H, this is negligible.

Case 2b: $r^* = r_{i^*}$ **but** $d^* \neq d_{i^*}$. If $r^* = r_{i^*}$, then the adversary has recovered the correct randomness r_{i^*} that was encrypted using Kyber and stored in the dictionary. This randomness was encrypted as:

$$(ct_{\mathtt{kyber},i^*}, K_{\mathtt{kem},i^*}) \leftarrow \mathsf{Kyber.Encaps}(\mathsf{ek}_{\mathtt{kyber},i^*}), \quad c_{i^*} = \mathsf{AES}_{K_{\mathtt{kem},i^*}}(r_{i^*}).$$

For the adversary to obtain r_{i^*} without knowing $\mathsf{dk}_{\mathtt{kyber},i^*}$, it must either 1) Break the IND-CCA2 security of Kyber to recover $K_{\mathtt{kem},i^*}$ from $ct_{\mathtt{kyber},i^*}$, or 2) Break the security of AES to recover r_{i^*} from c_{i^*} without $K_{\mathtt{kem},i^*}$.

However, even if the adversary has access to sk_{i^*} (as permitted in the robustness definition), it can compute r_{i^*} honestly via HSTE.PartDec. The key observation is that the partial decryption d_{i^*} is *deterministically computed* from sk_{i^*} and ct $d_{i^*} = ct_1 \cdot s_{i^*} \in R_q$, where s_{i^*} is party i^*'s BFV secret key and ct_1 is part of the ciphertext.

Since d_{i^*} is deterministic, if $r^* = r_{i^*}$ (ensuring verification passes) but $d^* \neq d_{i^*}$, then $\sigma^* = (i^*, d^*, r_{i^*})$ is *malformed*: it claims to be a partial decryption from party i^* (evidenced by the valid tag for r_{i^*}), but provides an incorrect BFV partial decryption $d^* \neq ct_1 \cdot s_{i^*}$.

But, the verification algorithm HSTE.PartVerify only checks the tag $H(i^* \| r^*)$, *not* the correctness of d^*. This is by design: verifying d^* would require knowledge of s_{i^*}, which is secret. Therefore, HSTE.PartVerify can accept a malformed σ^* with $d^* \neq d_{i^*}$ as long as $r^* = r_{i^*}$.

Game 3: Consider the decryption algorithm $\mathsf{HSTE.Dec}(ct, \{\sigma_i\}_{i \in \mathcal{S}})$ where $\mathcal{S}$ is the set of responding parties and $\sigma_{i^*} = (i^*, d^*, r_{i^*}) \in \{\sigma_i\}_{i \in \mathcal{S}}$ is the adversary's malformed partial decryption. The decryption algorithm computes $d_{\mathsf{agg}} = \frac{1}{m} \sum_{i \in \mathcal{S}_v} \lambda_i \cdot d_i$, where $\mathcal{S}_v = \{i \in \mathcal{S} : \mathsf{HSTE.PartVerify}(pk_i, ct, \sigma_i) = 1\}$ is the set of verified partial decryptions. If $\sigma^* = (i^*, d^*, r_{i^*})$ passes verification (which it does by Case 2b), then $i^* \in \mathcal{S}_v$ and d^*

contribute to $d_{\mathtt{agg}}$. The decryption becomes:

$$\begin{aligned}\tilde{\mu} &= ct_0 - d_{\mathtt{agg}} \\ &= ct_0 - \frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i \cdot d_i \\ &= ct_0 - \frac{1}{m}\left(\lambda_{i^*}\cdot d^* + \sum_{i\in\mathcal{S}_v\setminus\{i^*\}}\lambda_i\cdot d_i\right) \\ &= ct_0 - \frac{1}{m}\left(\lambda_{i^*}\cdot (d_{i^*}+\Delta d) + \sum_{i\in\mathcal{S}_v\setminus\{i^*\}}\lambda_i\cdot d_i\right),\end{aligned}$$

where $\Delta d := d^* - d_{i^*} \neq 0$ is the error introduced by the malformed partial decryption. From the correctness proof (Theorem 4), the honest decryption satisfies:

$$\tilde{\mu}_{\text{honest}} = ct_0 - \frac{1}{m}\sum_{i\in\mathcal{S}_v}\lambda_i\cdot d_i^{\text{honest}} = \Delta\cdot\mu + \mathsf{Err},$$

where Err is the BFV noise term with $\|\mathsf{Err}\|_\infty < q/(2p)$. With the malformed partial decryption $\tilde{\mu}_{\text{malformed}} = \tilde{\mu}_{\text{honest}} + \frac{\lambda_{i^*}}{m}\cdot\Delta d$. For the decryption to yield the correct message μ, we need:

$$\begin{aligned}\left\lfloor \frac{p}{q}\cdot\tilde{\mu}_{\text{malformed}} \right\rceil &= \left\lfloor \frac{p}{q}\cdot\left(\Delta\cdot\mu + \mathsf{Err} + \frac{\lambda_{i^*}}{m}\cdot\Delta d\right)\right\rceil \\ &= \left\lfloor \mu + \frac{p}{q}\cdot\mathsf{Err} + \frac{p}{q}\cdot\frac{\lambda_{i^*}}{m}\cdot\Delta d\right\rceil.\end{aligned}$$

This yields μ only if $\left|\frac{p}{q}\cdot\mathsf{Err} + \frac{p}{q}\cdot\frac{\lambda_{i^*}}{m}\cdot\Delta d\right| < \frac{1}{2}$. However, $\Delta d \in R_q$ is an arbitrary ring element chosen by the adversary. For robustness to fail, the adversary must:

1. Produce σ^* that passes verification ($r^* = r_{i^*}$), *and*
2. Choose $d^* \neq d_{i^*}$ such that the final decryption is *still incorrect*, i.e., $\mu' \neq \mu$.

If $|\mathcal{S}_v| \geq \tau = \lceil m/2 \rceil$ and at least $\tau - 1$ parties provide honest partial decryptions, then the Lagrange interpolation in $d_{\mathtt{agg}}$ amplifies the honest contributions. Specifically, for $|\mathcal{S}_v| = \tau$ and exactly one malformed partial decryption $\left|\frac{\lambda_{i^*}}{m}\right| = O\left(\frac{1}{m}\right)$, so the adversary's contribution is diluted by a factor $1/m$. For the adversary to cause decryption failure with non-negligible probability, it must choose Δd such that:

$$\|\Delta d\|_\infty \geq \Omega\left(\frac{q}{p}\right) \quad \text{(to overwhelm BFV noise)}.$$

But such large Δd would require the adversary to solve a BFV-related hard problem (since $d_{i^*} = ct_1 \cdot s_{i^*}$ is pseudorandom under RLWE).

We construct a reduction $\mathcal{B}_2$ that uses $\mathcal{A}$ to break the semantic security of BFV (Theorem 1) under RLWE: $\mathcal{B}_2$ receives a BFV challenge $(pk_{\mathtt{bfv}}, ct_{\mathtt{bfv}})$ where $ct_{\mathtt{bfv}} =$

(ct_0, ct_1), simulates the HSTE environment for $\mathcal{A}$, and obtains $(i^*, \sigma^* = (i^*, d^*, r^*))$. If σ^* passes verification and $d^* \neq d_{i^*}$, then $\mathcal{B}_2$ outputs $d^* - d_{i^*}$ as a distinguisher for the BFV challenge. Since $d_{i^*} = ct_1 \cdot s_{i^*}$ is pseudorandom under RLWE (by semantic security of BFV), producing a $d^* \neq d_{i^*}$ that meaningfully affects decryption with non-negligible probability contradicts the RLWE assumption. Therefore:

$$\Pr[\text{Case 2b leads to decryption failure}] \leq \mathsf{Adv}^{\mathsf{RLWE}}_{\mathcal{B}_2}(\lambda) \leq \mathsf{negl}(\lambda).$$

Combining Cases 2a and 2b:

$$\begin{aligned}\Pr[\mathsf{Win}_0] &\leq \Pr[\text{Case 2a}] + \Pr[\text{Case 2b leads to failure}] + |\Pr[\mathsf{Win}_1] - \Pr[\mathsf{Win}_0]| \\ &\leq \frac{Q_H^2}{2^{257}} + \mathsf{Adv}^{\mathsf{RLWE}}_{\mathcal{B}_2}(\lambda) + 2^{-128} + \mathsf{Adv}^{\mathsf{GA\text{-}StCDH}}_{\mathcal{B}_1}(\lambda) \\ &\leq \mathsf{negl}(\lambda).\end{aligned}$$

Thus, HSTE satisfies robustness (Definition 7). □

A.5 Details on Literature Review of Threshold Public Key Schemes

For further study of the community...

References

1. Torben P. Pedersen. A threshold cryptosystem without a trusted party. In Donald W. Davies, editor, *Advances in Cryptology — EUROCRYPT '91*, volume 547 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 1991.
2. Bo Qin, Qiang Wu, Lihua Zhang, and Josep Domingo-Ferrer. Threshold public-key encryption with adaptive security and short ciphertexts. In Miguel Soriano, Shi Qing, and Javier López, editors, *Information and Communications Security. ICICS 2010*, volume 6476 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2010.
3. Alice Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In Kai-Min Chung and Yuichi Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, volume 15484 of *Lecture Notes in Computer Science*. Springer, Singapore, 2025.
4. Antoine Urban and Martin Rambaud. Robust multiparty computation from threshold encryption based on rlwe. In Nicky Mouha and Nikolaos Nikiforakis, editors, *Information Security. ISC 2024*, volume 15257 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.
5. Karam Oudgoust and Paul Scholl. Simple threshold (fully homomorphic) encryption from lwe with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, volume 14438 of *Lecture Notes in Computer Science*. Springer, Singapore, 2023.
6. Joonsang Baek and Yuliang Zheng. Identity-based threshold decryption. In Feng Bao, Robert Deng, and Jianying Zhou, editors, *Public Key Cryptography – PKC 2004*, volume 2947 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2004.
7. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: attacks and defenses. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, SEC '24, USA, 2024. USENIX Association.

8. Christian Delerablée and David Pointcheval. Dynamic threshold public-key encryption. In David Wagner, editor, *Advances in Cryptology – CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2008.
9. Ai Ge, Rui Zhang, Chun Chen, Cong Ma, and Zhenfeng Zhang. Threshold ciphertext policy attribute-based encryption with constant size ciphertexts. In Willy Susilo, Yi Mu, and Jennifer Seberry, editors, *Information Security and Privacy. ACISP 2012*, volume 7372 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2012.
10. Hao Lin, Zhenfu Cao, Xiaolei Liang, and Jing Shao. Secure threshold multi authority attribute based encryption without a central authority. In Debasis R. Chowdhury, Vincent Rijmen, and Avishek Das, editors, *Progress in Cryptology - INDOCRYPT 2008*, volume 5365 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2008.
11. Ran Canetti and Shafi Goldwasser. An efficient threshold public key cryptosystem secure against adaptive chosen ciphertext attack (extended abstract). In Jacques Stern, editor, *Advances in Cryptology — EUROCRYPT '99*, volume 1592 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 1999.
12. Ronald Cramer, Ivan Damgård, and Jesper Buus Nielsen. Multiparty computation from threshold homomorphic encryption. In Birgit Pfitzmann, editor, *Advances in Cryptology — EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2001.
13. Sanjam Garg, Dimitris Kolonelos, G. V. Policharla, and Mingxun Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, volume 14926 of *Lecture Notes in Computer Science*. Springer, Cham, 2024.
14. Julien Devevey, Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Non-interactive cca2-secure threshold cryptosystems: Achieving adaptive security in the standard model without pairings. In Juan A. Garay, editor, *Public-Key Cryptography – PKC 2021*, volume 12710 of *Lecture Notes in Computer Science*. Springer, Cham, 2021.
15. Lukas Braun et al. An improved threshold homomorphic cryptosystem based on class groups. In Carmine Galdi and Duong H. Phan, editors, *Security and Cryptography for Networks. SCN 2024*, volume 14974 of *Lecture Notes in Computer Science*. Springer, Cham, 2024.
16. Lukas Braun, Ivan Damgård, and Claudio Orlandi. Secure multiparty computation from threshold encryption based on class groups. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, volume 14081 of *Lecture Notes in Computer Science*. Springer, Cham, 2023.
17. Ivan Damgård and Mads Jurik. A length-flexible threshold cryptosystem with applications. In Rei Safavi-Naini and Jennifer Seberry, editors, *Information Security and Privacy. ACISP 2003*, volume 2727 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2003.
18. Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2003.
19. Zhigang Xia, Xiaofei Yang, Min Xiao, and Debiao He. Provably secure threshold paillier encryption based on hyperplane geometry. In Joseph Liu and Ron Steinfeld, editors, *Information Security and Privacy. ACISP 2016*, volume 9723 of *Lecture Notes in Computer Science*. Springer, Cham, 2016.
20. Xiang Xie, Rui Xue, and Rui Zhang. Efficient threshold encryption from lossy trapdoor functions. In Byoung-Kyu Yang, editor, *Post-Quantum Cryptography. PQCrypto 2011*, volume 7071 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2011.

21. Kexin Xu, Benjamin Hong Meng Tan, Li-Ping Wang, Khin Mi Mi Aung, and Huaxiong Wang. Threshold homomorphic encryption from provably secure ntru. *The Computer Journal*, 66:2861–2873, 2023.
22. Willy Susilo, Guomin Yang, Fuchun Guo, and Qiong Huang. Constant-size ciphertexts in threshold attribute-based encryption without dummy attributes. *Information Sciences*, 429:349–360, 2018.
23. Xiaojun Zhang, Chunxiang Xu, Chunhua Jin, Run Xie, and Jining Zhao. Efficient fully homomorphic encryption from rlwe with an extension to a threshold encryption scheme. *Future Generation Computer Systems*, 36:180–186, 2014.
24. Yuta Sugizaki et al. Threshold fully homomorphic encryption over the torus. In Gene Tsudik, Mauro Conti, Kun Liang, and Georgios Smaragdakis, editors, *Computer Security – ESORICS 2023*, volume 14344 of *Lecture Notes in Computer Science*. Springer, Cham, 2024.
25. Guillaume Castagnos, Fabien Laguillaumie, and Isaac Tucker. Threshold linearly homomorphic encryption on $z/2^k z$. In Shweta Agrawal and Dong Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, volume 13792 of *Lecture Notes in Computer Science*. Springer, Cham, 2022.
26. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In David Pointcheval, editor, *Topics in Cryptology – CT-RSA 2006*, volume 3860 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2006.
27. Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hints: Threshold signatures with silent setup. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3034–3052, 2024.
28. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Tatsuaki Okamoto and Chris Peikert, editors, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer Berlin Heidelberg, 2010.
29. Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Mark Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for r1cs. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, volume 11476 of *Lecture Notes in Computer Science*, pages 257–287. Springer International Publishing, 2019.
30. Ronald Cramer and Victor Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In Jan Camenisch, editor, *Advances in Cryptology – EUROCRYPT 2002*, volume 2332 of *Lecture Notes in Computer Science*, pages 45–64. Springer Berlin Heidelberg, 2002.
31. Elette Boyle, Jonathan Katz, and Vinod Vaikuntanathan. Lossy trapdoor functions and their applications. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – CRYPTO 2016*, volume 9816 of *Lecture Notes in Computer Science*, pages 307–336. Springer Berlin Heidelberg, 2016.
32. Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In Huaxiong Li, Guomin Wang, and Jianping Cao, editors, *Public Key Cryptography – PKC 2007*, volume 4450 of *Lecture Notes in Computer Science*, pages 77–94. Springer Berlin Heidelberg, 2007.
33. Jan Camenisch and Victor Shoup. Practical verifiable encryption and decryption of discrete logarithms. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 126–144. Springer Berlin Heidelberg, 2003.

34. Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Simulation-sound arguments for lwe and applications to kdm-cca2 security. In Shiho Moriai and Nadia Heninger, editors, *Advances in Cryptology – ASIACRYPT 2018*, volume 11272 of *Lecture Notes in Computer Science*, pages 755–783. Springer International Publishing, 2018.
35. Guillaume Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from ddh. In Tal Malkin, editor, *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *Lecture Notes in Computer Science*, pages 257–274. Springer International Publishing, 2015.
36. Guillaume Castagnos, Fabien Laguillaumie, and Ivan Tucker. Practical fully secure unrestricted inner product functional encryption modulo p. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – ASIACRYPT 2018*, volume 11273 of *Lecture Notes in Computer Science*, pages 234–263. Springer Berlin Heidelberg, 2018.
37. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS 1987)*, pages 427–438, 1987.
38. Jonathan Bootle, Jens Groth, and Sebastian Jacques. Zero-knowledge arguments for unknown order groups with sublinear communication. In *Advances in Cryptology – EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 297–326. Springer Berlin Heidelberg, 2018.
39. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer Berlin Heidelberg, 1992.
40. Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In Shai Halevi and Danny Harnik, editors, *Public Key Cryptography – PKC 2006*, volume 3958 of *Lecture Notes in Computer Science*, pages 48–63. Springer Berlin Heidelberg, 2006.
41. Léo Ducas and Damien Stehlé. Sanitization of fhe ciphertexts. In Jean-Sébastien Coron and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 489–510. Springer Berlin Heidelberg, 2016.
42. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In Rei Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer Berlin Heidelberg, 2012.
43. Abhishek Jain, Peter M. R. Rasmussen, and Amit Sahai. Threshold fully homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2017*, volume 10210 of *Lecture Notes in Computer Science*, pages 301–331. Springer Berlin Heidelberg, 2017.
44. Hanjie Chen, Wei Dai, Min Kim, and Yongsoo Song. Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, pages 307–324. Association for Computing Machinery, 2019.
45. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS 1987)*, pages 427–438, 1987.
46. Guillaume Castagnos, Jean-François Génét, Thibault Labbé, Nicolas Liger, and Michele Oneto. Bandwidth-efficient threshold ec-dsa. In El Siam, Ahmed Seffen, and Abdallah Seffen, editors, *Public-Key Cryptography – PKC 2020*, volume 12000 of *Lecture Notes in Computer Science*, pages 307–337. Springer International Publishing, 2020.
47. Guillaume Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from ddh. In Tal Malkin, editor, *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *Lecture Notes in Computer Science*, pages 257–274. Springer International Publishing, 2015.

48. Guillaume Castagnos, Fabien Laguillaumie, and Ilya Tucker. Practical fully secure unrestricted inner product functional encryption modulo p. In Shiho Moriai and Nadia Heninger, editors, *Advances in Cryptology – ASIACRYPT 2018*, volume 11272 of *Lecture Notes in Computer Science*, pages 128–158. Springer International Publishing, 2018.
49. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, volume 10993 of *Lecture Notes in Computer Science*, pages 295–325. Springer Berlin Heidelberg, 2018.
50. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3):13:1–13:46, 2014.
51. Shweta Bai, Ivan Damgård, Craig Gentry, and Vinod Vaikuntanathan. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. In Jean-Sébastien Coron and Mogens Nielson, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10821 of *Lecture Notes in Computer Science*, pages 149–178. Springer International Publishing, 2018.
52. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In Jan Camenisch, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer Berlin Heidelberg, 2001.
53. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
54. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In Jan Camenisch, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer Berlin Heidelberg, 2001.
55. David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*, pages 89–105. Springer Berlin Heidelberg, 1992.
56. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer Berlin Heidelberg, 2010.
57. Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, 1991.
58. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
59. Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Efficient pairings on supersingular and barreto-naehrig curves. In Xuejia Lai, editor, *Advances in Cryptology – ASIACRYPT 2003*, volume 2894 of *Lecture Notes in Computer Science*, pages 259–275. Springer Berlin Heidelberg, 2003.
60. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT '99*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238. Springer Berlin Heidelberg, 1999.
61. David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*, pages 89–105. Springer Berlin Heidelberg, 1992.
62. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
63. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer Berlin Heidelberg, 2001.
64. Cyrille Delerablée, Pascal Paillier, and David Pointcheval. Fully collusion secure dynamic broadcast encryption with constant-size ciphertexts or decryption keys. In

Tsuyoshi Takagi, Tatsuaki Okamoto, Eiji Okamoto, and Takeshi Okamoto, editors, *Pairing-Based Cryptography – Pairing 2007*, volume 4575 of *Lecture Notes in Computer Science*, pages 39–59. Springer Berlin Heidelberg, 2007.
65. International Organization for Standardization. Information technology – security techniques – encryption algorithms – part 2: Asymmetric ciphers. ISO/IEC 18033-2:2006, 2006. Final Committee Draft from December 2004 was a precursor.
66. Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS 1987)*, pages 427–438, 1987.
67. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO '89*, volume 435 of *Lecture Notes in Computer Science*, pages 307–315. Springer New York, 1990.
68. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer Berlin Heidelberg, 1992.
69. Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 207–220. Springer Berlin Heidelberg, 2000.
70. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
71. Ivan Damgård and Mads Jurik. A generalisation, a simplification and some applications of paillier's probabilistic public-key system. In Joe Kilian and Tal Rabin, editors, *Public Key Cryptography – PKC 2001*, volume 1992 of *Lecture Notes in Computer Science*, pages 119–136. Springer Berlin Heidelberg, 2001.
72. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo G. Desmedt, editor, *Advances in Cryptology – CRYPTO '94*, volume 839 of *Lecture Notes in Computer Science*, pages 174–187. Springer Berlin Heidelberg, 1994.
73. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT '99*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238. Springer Berlin Heidelberg, 1999.
74. Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 331–350. Springer Berlin Heidelberg, 2001.
75. Anna Lysyanskaya and Chris Peikert. Adaptive security in the threshold setting: From cryptosystems to signature schemes. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 331–350. Springer Berlin Heidelberg, 2001.
76. Guillaume Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from ddh. In Tal Malkin, editor, *Topics in Cryptology – CT-RSA 2015*, volume 9048 of *Lecture Notes in Computer Science*, pages 257–274. Springer International Publishing, 2015.
77. Guillaume Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from ddh. In Tal Malkin, Vladimir Kolesnikov, Amy B. Lewko, and Michalis Polychronakis, editors, *Applied Cryptography and Network Security*, volume 9092 of *Lecture Notes in Computer Science*, pages 481–498. Springer International Publishing, 2015.
78. Ivan Damgård and Rune Thorbek. Linear integer secret sharing and distributed exponentiation. In Serge Vaudenay, editor, *Public Key Cryptography – PKC 2006*, volume 3958 of *Lecture Notes in Computer Science*, pages 48–63. Springer Berlin Heidelberg, 2006.

79. Ran Canetti and Yehuda Lindell. Security of multi-party protocols in the universal composability framework. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 1–20. Springer Berlin Heidelberg, 2001.
80. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
81. Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *Advances in Cryptology – CRYPTO '98*, volume 1462 of *Lecture Notes in Computer Science*, pages 13–25. Springer Berlin Heidelberg, 1998.
82. Ralph Merkle. Secure communications over insecure channels. In Ralph Merkle, editor, *Proceedings of the Workshop on Computer Science*, pages 235–248. Stanford University, 1979.
83. Dan Boneh, Xavier Boyen, and Eu-Jin Goh. Hierarchical identity based encryption with constant size ciphertext. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 440–456. Springer Berlin Heidelberg, 2005.
84. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
85. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473. Springer Berlin Heidelberg, 2005.
86. Chen Chen, Zhen Zhang, and Dengguo Feng. Efficient ciphertext policy attribute-based encryption with constant-size ciphertext and constant computation-cost. In Xavier Boyen and Xiaofeng Chen, editors, *Provable Security*, volume 6980 of *Lecture Notes in Computer Science*, pages 84–101. Springer Berlin Heidelberg, 2011.
87. Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2007*, volume 4515 of *Lecture Notes in Computer Science*, pages 293–310. Springer Berlin Heidelberg, 2007.
88. Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Robust threshold dss. In Birgit Pfitzmann, editor, *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 152–169. Springer Berlin Heidelberg, 2001.
89. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer Berlin Heidelberg, 2001.
90. Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 514–532. Springer Berlin Heidelberg, 2001.
91. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
92. G. R. Blakley. Safeguarding cryptographic keys. In *Proceedings of the National Computer Conference*, volume 48, pages 313–317. AFIPS Press, 1979.
93. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT '99*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238. Springer Berlin Heidelberg, 1999.
94. David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO '92*, volume 740 of *Lecture Notes in Computer Science*, pages 89–105. Springer Berlin Heidelberg, 1993.
95. Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, STOC '08, pages 187–196. Association for Computing Machinery, 2008.

96. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
97. Payman Mohassel. One-time signatures and chameleon hash functions. In Alex Biryukov, Guang Gong, and Douglas R. Stinson, editors, *Selected Areas in Cryptography*, volume 6544 of *Lecture Notes in Computer Science*, pages 302–319. Springer Berlin Heidelberg, 2011.
98. Kexin Xu, Benjamin Hong Meng Tan, Li-Ping Wang, Khin Mi Mi Aung, and Huaxiong Wang. Threshold homomorphic encryption from provably secure ntru. *The Computer Journal*, 66:2861–2873, 2023.
99. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer Berlin Heidelberg, 2010.
100. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, volume 10993 of *Lecture Notes in Computer Science*, pages 295–325. Springer Berlin Heidelberg, 2018.
101. Gilad Asharov, Yehuda Lindell, and Aviad Padan. Multiparty computation with low communication and computation. In Rei Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 617–635. Springer Berlin Heidelberg, 2012.
102. Donald Beaver. Commodity-based cryptography. In *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, STOC '97, pages 446–455. Association for Computing Machinery, 1997.
103. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC '88, pages 1–10. Association for Computing Machinery, 1988.
104. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. Tfhe: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.
105. Dan Boneh, Xavier Boyen, and Eu-Jin Goh. Hierarchical identity based encryption with constant size ciphertext. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 440–456. Springer Berlin Heidelberg, 2005.
106. Cyrille Delerablée and David Pointcheval. Dynamic broadcast encryption from bilinear maps. In Phong Q. Nguyen and Pascal Paillier, editors, *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 617–635. Springer Berlin Heidelberg, 2008.
107. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
108. Dan Boneh and Xavier Boyen. Efficient selective-id secure identity based encryption without random oracles. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 223–238. Springer Berlin Heidelberg, 2004.
109. Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 255–272. Springer Berlin Heidelberg, 2004.
110. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.

111. Zvika Brakerski, Adeline Langlois, Chris Peikert, Damien Stehlé, and Vinod Vaikuntanathan. Worst-case to average-case reductions for module lattices. In *International Colloquium on Automata, Languages, and Programming (ICALP) 2012*, volume 7392 of *Lecture Notes in Computer Science*, pages 110–121. Springer, Berlin, Heidelberg, 2012.
112. Vadim Lyubashevsky. Fewer keys and more efficiency from the ring-lwe cryptosystem. In *Annual International Cryptology Conference*, volume 7417 of *Lecture Notes in Computer Science*, pages 172–190. Springer, Berlin, Heidelberg, 2012. Presented at EUROCRYPT 2012, but formally introduces the module setting and is the basis for its adoption in subsequent schemes.
113. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer Berlin Heidelberg, 2010.
114. Dan Boneh, Xavier Boyen, and Hovav Shacham. Short signatures from the Gap Diffie-Hellman group. In *Advances in Cryptology – ASIACRYPT 2004*, volume 3329 of *Lecture Notes in Computer Science*, pages 514–532. Springer, Berlin, Heidelberg, 2004.
115. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 565–596, Cham, 2018. Springer International Publishing.
116. Ralph C. Merkle. Secure communications over insecure channels. In Ralph C. Merkle, editor, *Proceedings of the Workshop on Computer Science*, pages 235–248. Stanford University, 1979.
117. Zvika Brakerski, Yuval Fan, and Frederik Vercauteren. Fully homomorphic encryption with a leveled structure. In *Proceedings of the 5th International Conference on Pairing-Based Cryptography*, volume 7281 of *Lecture Notes in Computer Science*, pages 149–166. Springer Berlin Heidelberg, 2012.
118. Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) lwe. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, 2011.
119. Joppe Bos, Daniel Costello, Léo Ducas, Edward Mella, Christian Panny, Thomas Pöppelmann, Thomas Prest, William Schanck, Peter Schneider, and Peter Schwabe. Crystals – kyber: A high-speed proof-of-concept implementation of module-lwe key exchange. In *Advances in Cryptology – ASIACRYPT 2017*, volume 10625 of *Lecture Notes in Computer Science*, pages 353–389. Springer, Cham, 2017.
120. Pavan Sanal, Emre Karagoz, Hyesoo Seo, Reza Azarderakhsh, and Mehran Mozaffari-Kermani. Kyber on arm64: Compact implementations of kyber on 64-bit arm cortex-a processors. In Jorge Garcia-Alfaro, Shujun Li, Radha Poovendran, Hervé Debar, and Moti Yung, editors, *Security and Privacy in Communication Networks. SecureComm 2021*, volume 399 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*. Springer, Cham, 2021.
121. Takeshi Saito, Keisuke Xagawa, and Takashi Yamakawa. Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In Jesper Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, volume 10822 of *Lecture Notes in Computer Science*. Springer, Cham, 2018.
122. Wouter Castryck, Tanja Lange, Chris Martindale, Lorenz Panny, Joost Renes, and Frederik Vercauteren. CSIDH: An efficient post-quantum key exchange from commutative group actions. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11273 of *Lecture Notes in Computer Science*, pages 3–28. Springer, Cham, 2018.

123. Johannes Duman, Dennis Hartmann, Eike Kiltz, Simon Kunzweiler, Julian Lehmann, and Daniel Riepel. Group action key encapsulation and non-interactive key exchange in the qrom. In Shweta Agrawal and Dong Lin, editors, *Advances in Cryptology – ASIACRYPT 2022*, volume 13792 of *Lecture Notes in Computer Science*. Springer, Cham, 2022.
124. Ragnar Groot Koerkamp. PtrHash: Minimal Perfect Hashing at RAM Throughput. In Petra Mutzel and Nicola Prezza, editors, *23rd International Symposium on Experimental Algorithms (SEA 2025)*, volume 338 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
125. Mesut F. Esgin, Ngoc Khanh Nguyen, and Gabor Seiler. Practical exact proofs from lattices: New techniques to exploit fully-splitting rings. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, volume 12492 of *Lecture Notes in Computer Science*. Springer, Cham, 2020.
126. Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In *Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part VII*, page 352–386, Berlin, Heidelberg, 2024. Springer-Verlag.